

## Indice

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>Lista di argomenti base per cominciare Spring.....</b>                | <b>7</b>  |
| Roadmap - Spring Boot Mastery (Guida su GitHub).....                     | 7         |
| • Step 1: Master Java Fundamentals.....                                  | 7         |
| • Step 2: Learn Maven or Gradle.....                                     | 7         |
| • Step 3: Dive Into Spring Core.....                                     | 7         |
| • Step 4: Build with Spring Boot.....                                    | 7         |
| • Step 5: Build Secure REST APIs.....                                    | 8         |
| • Step 6: Secure Your Applications.....                                  | 8         |
| • Step 7: Work with Databases.....                                       | 8         |
| • Step 8: Testing Best Practices.....                                    | 8         |
| • Step 9: Connect to External APIs.....                                  | 8         |
| • Step 10: Advanced Topics.....                                          | 9         |
| Cos'è una REST api?.....                                                 | 10        |
| Smontiamo l'acronimo parola per parola.....                              | 10        |
| REpresentational (Rappresentazione).....                                 | 10        |
| State (Stato).....                                                       | 10        |
| Transfer (Trasferimento).....                                            | 10        |
| Principi Architetture e Vincoli (RESTful).....                           | 11        |
| <b>The ULTIMATE Guide to Spring Boot: Spring Boot for Beginners.....</b> | <b>12</b> |
| Quickstart App.....                                                      | 12        |
| Spring Initializr.....                                                   | 12        |
| Cosa troveremo nell'IDE.....                                             | 13        |
| Implementazione del codice iniziale.....                                 | 13        |
| Maven.....                                                               | 15        |
| Perché è fondamentale in un progetto?.....                               | 15        |
| Maven Wrapper (mvnw).....                                                | 15        |
| Cicli di Vita e Fasi (Lifecycles & Phases).....                          | 15        |
| La Struttura Standard delle Directory.....                               | 16        |
| Importanza della Convenzione.....                                        | 16        |
| Workflow di Sviluppo Consigliato.....                                    | 17        |
| Metodo Alternativo: Spring Boot Maven Plugin.....                        | 17        |
| Spring Framework & Boot.....                                             | 18        |
| Spring Framework vs Spring Boot.....                                     | 18        |
| Spring App Layers.....                                                   | 18        |
| 1. Livello di Persistenza (Persistence Layer).....                       | 18        |
| 2. Livello di Servizio (Service Layer).....                              | 18        |
| 3. Livello di Presentazione (Presentation Layer).....                    | 19        |
| Modularity.....                                                          | 19        |
| Dependency Injection.....                                                | 20        |
| Inversion of Control (IoC).....                                          | 20        |
| Il Problema: Accoppiamento Forte.....                                    | 20        |
| La Soluzione: Inversione del Controllo.....                              | 20        |
| Vantaggi della Modularità.....                                           | 20        |

|                                                         |    |
|---------------------------------------------------------|----|
| IoC vs. Dependency Injection.....                       | 20 |
| Introducing Beans.....                                  | 21 |
| Definizione di "Bean".....                              | 21 |
| Il concetto di "Application Context".....               | 21 |
| Metodi per dichiarare i Beans.....                      | 21 |
| Vantaggio Pratico: Swap delle Dipendenze.....           | 21 |
| @Component and Friends.....                             | 26 |
| L'annotazione @Component.....                           | 26 |
| Gli "Amici" di @Component (Stereotipi).....             | 26 |
| Esempio di flessibilità.....                            | 26 |
| Component Scanning.....                                 | 27 |
| Definizione e Avvio.....                                | 27 |
| Il Processo di Registrazione.....                       | 27 |
| Gerarchia e Punto di Partenza.....                      | 27 |
| Il Ruolo di @SpringBootApplication.....                 | 27 |
| @SpringBootApplication Annotation.....                  | 28 |
| Una "Super Annotazione" (Alias).....                    | 28 |
| Il Punto di Ingresso (Entry Point).....                 | 28 |
| Autoconfiguration.....                                  | 29 |
| Cos'è l'Autoconfigurazione?.....                        | 29 |
| Il Ruolo degli "Starters".....                          | 29 |
| Come Funziona il Meccanismo.....                        | 29 |
| Esempio Pratico: DispatcherServlet.....                 | 29 |
| Configuration.....                                      | 31 |
| Configuration files.....                                | 31 |
| Scopo dei file di configurazione.....                   | 31 |
| Formati supportati: Properties e YAML.....              | 31 |
| Documentazione delle proprietà comuni.....              | 31 |
| Configurazione per i Test.....                          | 31 |
| Environment Variables.....                              | 32 |
| Importanza delle Variabili d'Ambiente.....              | 32 |
| Regole di Conversione (Naming Convention).....          | 32 |
| Modalità di Utilizzo.....                               | 32 |
| Ordine di Priorità.....                                 | 32 |
| Configuration Properties.....                           | 33 |
| Cos'è la Configuration Properties?.....                 | 33 |
| L'annotazione @ConfigurationProperties e il prefix..... | 33 |
| Requisiti e Struttura della Classe.....                 | 33 |
| Abilitazione e Iniezione.....                           | 33 |
| Esempio Pratico: PizzaConfig.....                       | 34 |
| Databases Part 1 - Basics.....                          | 38 |
| Database Layers.....                                    | 38 |
| Database Driver.....                                    | 38 |
| JDBC (Java Database Connectivity).....                  | 38 |

|                                                    |    |
|----------------------------------------------------|----|
| Spring JDBC.....                                   | 38 |
| JPA (Java Persistence API) e Hibernate.....        | 38 |
| Spring Data JPA.....                               | 39 |
| Connect to a H2 Database.....                      | 40 |
| Caratteristiche di H2.....                         | 40 |
| Configurazione e Dipendenze.....                   | 40 |
| La "Magia" dell'Autoconfigurazione.....            | 40 |
| Configurazione Manuale (Opzionale).....            | 40 |
| Verifica della Connessione.....                    | 40 |
| Connect to a PostgreSQL Database.....              | 42 |
| Dipendenze necessarie.....                         | 42 |
| Configurazione del Database (Docker Compose).....  | 42 |
| Configurazione in application.properties.....      | 42 |
| Gestione degli errori e Avvio.....                 | 42 |
| Verifica della connessione.....                    | 43 |
| Initialise DB Schema.....                          | 45 |
| I file di inizializzazione SQL.....                | 45 |
| Configurazione in application.properties.....      | 46 |
| Workflow di esecuzione.....                        | 46 |
| Verifica dei risultati.....                        | 46 |
| Databases Part 2 - Spring JDBC (SQL).....          | 47 |
| JdbcTemplate Setup.....                            | 47 |
| Cos'è JdbcTemplate?.....                           | 47 |
| Creazione della Classe di Configurazione.....      | 47 |
| Definizione del Bean JdbcTemplate.....             | 48 |
| Utilizzo tramite Dependency Injection.....         | 48 |
| What is a DAO?.....                                | 49 |
| Il Problema: Duplicazione e Accoppiamento.....     | 49 |
| La Soluzione: Il Pattern DAO.....                  | 49 |
| Esempio Pratico: Libri e Autori.....               | 49 |
| Vantaggi Architetture.....                         | 49 |
| Set Up a DAO.....                                  | 50 |
| Definizione dello Schema SQL.....                  | 50 |
| Creazione degli Oggetti di Dominio (Entities)..... | 51 |
| Struttura del Package DAO.....                     | 51 |
| Implementazione e Iniezione di JdbcTemplate.....   | 52 |
| Create DAOs.....                                   | 53 |
| Sviluppo tramite Unit Test.....                    | 53 |
| Implementazione per l'Autore (AuthorDao).....      | 54 |
| Implementazione per il Libro (BookDao).....        | 55 |
| Sicurezza e SQL Injection.....                     | 56 |
| Implement Read One.....                            | 57 |
| Il concetto di "Read One".....                     | 57 |
| L'utilizzo di RowMapper.....                       | 57 |

|                                                          |    |
|----------------------------------------------------------|----|
| Esecuzione della Query con JdbcTemplate.....             | 58 |
| Dettaglio SQL e Sicurezza.....                           | 59 |
| Find Many.....                                           | 60 |
| Il Concetto di "Find Many".....                          | 60 |
| Test Unitario e Implementazione.....                     | 60 |
| Test di Integrazione.....                                | 60 |
| Gestione della "Test Pollution".....                     | 60 |
| Full Update.....                                         | 62 |
| Definizione di "Full Update".....                        | 62 |
| Implementazione per l'Autore.....                        | 62 |
| Implementazione per il Libro.....                        | 62 |
| Verifica tramite Test.....                               | 62 |
| Implement Delete.....                                    | 64 |
| Implementazione per l'Autore (AuthorDao).....            | 64 |
| Implementazione per il Libro (BookDao).....              | 64 |
| Verifica e Testing.....                                  | 64 |
| Verso l'automazione: JPA e Hibernate.....                | 65 |
| Create Integration Test.....                             | 66 |
| Obiettivo del Test di Integrazione.....                  | 66 |
| Configurazione con H2 (In-Memory Database).....          | 66 |
| Iniezione delle Dipendenze nei Test.....                 | 66 |
| Gestione della "Test Pollution" con @DirtiesContext..... | 66 |
| Flusso di Esecuzione: Create & Recall.....               | 66 |
| Databases Part 3 - Spring Data JPA (Objects).....        | 73 |
| Spring Data JPA Setup.....                               | 73 |
| Rimozione del codice "Boilerplate".....                  | 73 |
| Aggiornamento delle dipendenze nel pom.xml.....          | 73 |
| Introduzione dei Repository.....                         | 74 |
| Create Entities.....                                     | 75 |
| Trasformazione della classe Author.....                  | 75 |
| Trasformazione della classe Book.....                    | 76 |
| Mappatura delle Relazioni (Many-to-One).....             | 76 |
| Vantaggi dell'approccio a Entità.....                    | 77 |
| Hibernate Auto DDL.....                                  | 78 |
| Il ruolo di Hibernate Auto DDL.....                      | 78 |
| Configurazione in application.properties.....            | 78 |
| Valori della proprietà.....                              | 78 |
| Risultato Pratico.....                                   | 79 |
| Implement Create & Read.....                             | 80 |
| Ereditarietà da CrudRepository.....                      | 80 |
| Operazione di Creazione: il metodo save().....           | 80 |
| Lettura Singola: findById().....                         | 80 |
| Verifica senza SQL.....                                  | 81 |
| Implement Find All.....                                  | 82 |

|                                                         |    |
|---------------------------------------------------------|----|
| Il Metodo findAll().....                                | 82 |
| Automazione Totale.....                                 | 82 |
| Test di Integrazione.....                               | 82 |
| Implement Update.....                                   | 84 |
| Il Metodo "Polimorfico" save().....                     | 84 |
| Workflow di Aggiornamento per l'Autore.....             | 84 |
| Workflow per il Libro.....                              | 85 |
| Vantaggi Architetture.....                              | 85 |
| Implement Delete.....                                   | 86 |
| Il Metodo deleteById().....                             | 86 |
| Opzioni di Cancellazione.....                           | 86 |
| Workflow di Test e Verifica.....                        | 86 |
| Semplicità rispetto al DAO.....                         | 87 |
| Custom Queries.....                                     | 88 |
| Query Methods (Derived Queries).....                    | 88 |
| HQL (Hibernate Query Language).....                     | 89 |
| Definizione e Differenza con SQL.....                   | 89 |
| Implementazione con @Query.....                         | 89 |
| Esempio di Sintassi e Parametri.....                    | 89 |
| Automazione del Framework.....                          | 90 |
| Jackson & JSON.....                                     | 91 |
| What is Jackson?.....                                   | 91 |
| Definizione e Ruolo.....                                | 91 |
| Concetti Base: Marshalling e Unmarshalling.....         | 91 |
| Integrazione in Spring Web.....                         | 91 |
| Lo strumento principale: ObjectMapper.....              | 92 |
| Personalizzazione tramite Annotazioni.....              | 92 |
| Approfondimento - Testing - Postman e Bruno.....        | 93 |
| Il flusso di testing con Postman.....                   | 93 |
| Bruno: Un'alternativa moderna e Open Source.....        | 94 |
| Jackson & Spring Web.....                               | 95 |
| Integrazione Automatica.....                            | 95 |
| Funzionamento nelle Richieste GET (Marshalling).....    | 96 |
| Funzionamento nelle Richieste POST (Unmarshalling)..... | 96 |
| Semplicità del Codice.....                              | 96 |
| Java Objects to JSON.....                               | 97 |
| Istanza di ObjectMapper.....                            | 97 |
| Il Metodo writeValueAsString.....                       | 97 |
| Gestione delle Eccezioni.....                           | 98 |
| Mappatura dei Campi.....                                | 98 |
| Verifica nei Test.....                                  | 98 |
| JSON to Java Objects.....                               | 99 |
| Il Metodo readValue().....                              | 99 |
| Requisiti della Classe Java.....                        | 99 |

|                                                 |     |
|-------------------------------------------------|-----|
| Logica di Mappatura e Convenzioni.....          | 100 |
| Gestione degli Errori e Campi Sconosciuti.....  | 100 |
| Renaming JSON Properties.....                   | 101 |
| La convenzione di default.....                  | 101 |
| L'annotazione @JsonProperty.....                | 101 |
| Effetto Bidirezionale.....                      | 101 |
| Integrazione con Spring Web.....                | 101 |
| Ignore Properties.....                          | 102 |
| Il problema: UnrecognizedPropertyException..... | 102 |
| La soluzione: @JsonIgnoreProperties.....        | 102 |
| Configurazione ignoreUnknown = true.....        | 102 |

# Lista di argomenti base per cominciare Spring

Nonostante il documento sia stato costruito come trascrizione del video di [Aaron Bourne](#), cercando in giro mi sono imbattuto in una roadmap molto utile per quanto riguarda il come approcciare a questa tecnologia, soprattutto per quanto riguarda persone che non si sono mai interfacciate con la programmazione.

## Roadmap - Spring Boot Mastery (Guida su GitHub)

- Una roadmap gratuita per sapere esattamente che cosa imparare e in che ordine: Java, REST API, Spring Security, accesso DB, testing, microservizi, Docker, ecc. ([GitHub](#)). Di seguito tutti i punti riportati:

- **Step 1: Master Java Fundamentals**

Before diving into Spring Boot, strengthen your Java basics:

- Java 17 or 21 (Newer Versions)
- OOP (Object-Oriented Programming)
- [Lambdas](#) and Streams
- Multithreading and Concurrency

- **Step 2: Learn Maven or Gradle**

Understand build tools to manage dependencies and deployment:

- Dependency Management
- Build Profiles (Dev vs Prod)
- Packaging and Deployment

- **Step 3: Dive Into Spring Core**

Build a foundation with the Spring Framework:

- Dependency Injection (DI)
- Inversion of Control (IoC)
- Bean Lifecycle
- Configuration Classes
- Know what JSON is

- **Step 4: Build with Spring Boot**

Spring Boot accelerates app development:

- Auto Configuration
- Starter Dependencies
- Profiles and Properties
- Monitoring with Actuator

- Step 5: Build Secure REST APIs

Create production-grade REST APIs:

- CRUD with:
  - JPA / Hibernate
  - PostgreSQL / MySQL
- Controllers and DTOs
- Data Validation
- Exception Handling
- Rate Limiting

- Step 6: Secure Your Applications

Security is a must:

- Spring Security Fundamentals
- JWT Authentication
- OAuth2
- Custom Security Filters

- Step 7: Work with Databases

Spring Boot integrates well with databases:

- Spring Data JPA
- Custom Queries
- Transaction Management
- Migrations with Flyway/Liquibase

- Step 8: Testing Best Practices

Testing ensures reliability and maintainability:

- Unit Testing with JUnit 5
- Mocking with Mockito
- Integration Testing with Testcontainers

- Step 9: Connect to External APIs

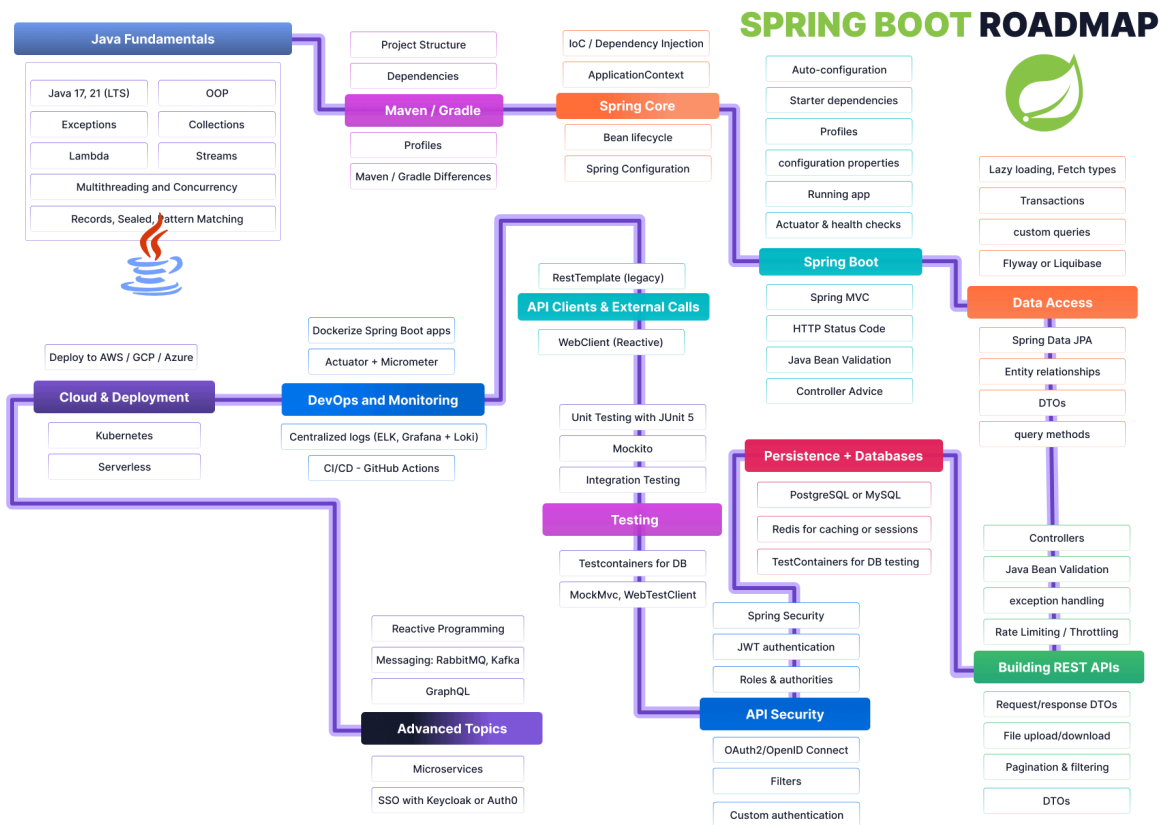
Integrate with third-party APIs:

- Use WebClient for non-blocking communication
- Use RestTemplate for legacy support

## ● Step 10: Advanced Topics

Take your backend expertise to the next level:

- Microservices Architecture
- Messaging with Kafka
- GraphQL APIs with Spring
- Reactive Programming (WebFlux)
- Cloud Deployment (AWS)
- Containerization with Docker
- Perhaps some basics of React or Angular for frontend



# Cos'è una REST api?

In questo file troveremo molto spesso la parola **REST**, concetto che se cercato online confonde e viene descritto in modo molto (ritengo troppo) logorroico.

Una **REST API** (Representational State Transfer Application Programming Interface) è un'interfaccia di programmazione che definisce **un insieme di regole, vincoli e convenzioni architetturali** utilizzate per consentire la comunicazione e lo scambio di dati tra due sistemi software distinti (tipicamente un client e un server) attraverso il protocollo di rete **HTTP**.

A differenza delle applicazioni web tradizionali, una REST API opera esclusivamente sul livello logico dei dati. Essa non si occupa della resa grafica (interfaccia utente), ma espone risorse informative in un formato testuale standardizzato e agnostico rispetto alla piattaforma, di norma **JSON** (JavaScript Object Notation) o **XML**.

## Smontiamo l'acronimo parola per parola

### REpresentational (Rappresentazione)

Il server (il backend) custodisce i dati veri e propri dentro a un database (es. righe e colonne di una tabella MySQL). Quando il frontend chiede quei dati, il server non gli dà accesso al database. Crea invece un documento di testo (quasi sempre un **JSON**) che contiene la *fotografia descrittiva* di quei dati.

- Il JSON è la **rappresentazione** digitale della risorsa.

### State (Stato)

Lo "Stato" è la situazione in cui si trovano i dati in quel millesimo di secondo in cui fai la richiesta.

- Se un conto bancario ha 50€, lo *stato* attuale è 50€.
- Se un utente ha appena cambiato password, lo *stato* della risorsa utente è aggiornato con la nuova password.  
L'API cattura questo stato e lo impacchetta.

### Transfer (Trasferimento)

È l'azione fisica di spostare quel pacchetto di dati (la rappresentazione dello stato) attraverso la rete internet, dal computer in cui risiede il server fino al dispositivo dell'utente (smartphone, PC, browser), usando le strade standard del protocollo HTTP.

## Principi Architetture e Vincoli (RESTful)

Talvolta, viene anche espresso il termine **RESTful**, in parallelo con REST ed in maniera erratamente intercambiabile.

Affinché un'API sia definita RESTful, l'infrastruttura di backend deve rispettare rigorosamente i seguenti criteri di progettazione:

- **Disaccoppiamento Client-Server:** La logica dei dati (Backend) e l'interfaccia utente (Frontend) sono separate. Il server non ha conoscenza di come i dati verranno visualizzati; il client non ha conoscenza di come i dati vengano memorizzati o elaborati.
- **Statelessness (Assenza di stato):** Ogni richiesta inviata dal client al server deve essere autocontenuta. Il server non memorizza sessioni o contesti storici dell'utente. Ogni chiamata deve includere tutte le informazioni necessarie per la sua corretta elaborazione e autenticazione.
- **Interfaccia Uniforme:** Le risorse del sistema sono identificate in modo univoco tramite stringhe chiamate **URI/URL** (es. `/api/v1/utenti`). L'interazione con esse avviene attraverso i metodi nativi del protocollo HTTP.
- **Capacità di Caching:** Le risposte del server devono esplicitare se i dati inviati possono essere memorizzati nella cache del client, riducendo il carico di rete e migliorando le prestazioni complessive.

Per informazioni più approfondite riguardo le differenze tra una REST API e una RESTful API vi è un ottimo approfondimento all'interno dell'articolo [Difference Between REST API and RESTful API](#).

# The ULTIMATE Guide to Spring Boot: Spring Boot for Beginners

Qui tratteremo un corso approfondito su **Spring Boot** per principianti, focalizzato sulla creazione di una **REST API** completa integrata con un database **PostgreSQL**. La guida illustra l'intero ciclo di sviluppo, partendo dalla configurazione iniziale tramite **Spring Initializr** e l'utilizzo di strumenti essenziali come **Maven** e **IntelliJ IDEA**. Vengono spiegati concetti fondamentali quali l'**inversione del controllo**, la **dependency injection** e l'uso delle annotazioni per la gestione dei **Bean**. L'autore approfondisce inoltre lo strato di persistenza dei dati, confrontando l'uso di **JDBC** e **JPA** per l'interazione con il database. Infine, il percorso formativo include istruzioni pratiche per il confezionamento dell'applicazione in un contenitore **Docker** e la successiva distribuzione sul cloud tramite **AWS**.

## Quickstart App

La costruzione dell'applicazione segue un processo guidato che parte dalla configurazione esterna fino alla scrittura del codice:

### Spring Initializr

Lo [Spring Initializr](https://start.spring.io) è un sito web (raggiungibile all'indirizzo **start.spring.io**) che rappresenta il punto di partenza per la creazione di un'applicazione Spring Boot. Il suo scopo principale è permettere agli sviluppatori di **configurare e scaricare un progetto "skeleton"** (un'ossatura di base) già pronto per essere utilizzato.

Attraverso questa interfaccia, è possibile definire diverse impostazioni fondamentali del progetto:

- **Gestione delle dipendenze e linguaggio:** Si può scegliere tra strumenti come **Maven** o Gradle e selezionare il linguaggio di programmazione tra **Java**, Groovy o Kotlin.
- **Versione di Spring Boot:** Consente di selezionare la versione del framework desiderata, solitamente optando per l'ultima versione stabile disponibile.
- **Metadati del progetto:** Permette di inserire informazioni identificative come il **Group** (ad esempio il dominio dell'azienda) e l'**Artifact** (il nome del progetto), in questo caso: Group `com.dev tiro` e Artifact `quickstart`.
- **Packaging e Java:** Si può scegliere il tipo di confezionamento del software (come un file **JAR** o **WAR**) e la versione di Java installata sulla propria macchina.
- **Dipendenze:** È possibile aggiungere moduli specifici necessari al progetto, come **Spring Web** per la creazione di API REST.

Una volta completata la configurazione e cliccato sul tasto "Generate", lo strumento fornisce i **componenti essenziali** (Bare Bones) dell'applicazione e un test di base, che possono essere importati in un ambiente di sviluppo (IDE) per iniziare a programmare.

## Cosa troveremo nell'IDE

- **Il Ruolo di Spring Web:** Questa dipendenza permette di trasformare semplici classi Java in API REST con pochissimo codice. I controller possono restituire stringhe semplici o oggetti complessi in formato **JSON**.
- **Classe d'ingresso (Entry Point):** La classe `QuickstartApplication` è il punto di partenza. Contiene il metodo `main` e l'annotazione `@SpringBootApplication`. Quest'ultima è un alias per diverse annotazioni che identificano l'app e attivano la configurazione automatica.
- **Struttura delle Cartelle:**
  - `src/main/resources`: Contiene il file `application.properties`, usato per personalizzare la configurazione dell'app (come cambiare la porta del server), si possono avere anche `application.yaml` se preferiamo utilizzare uno yaml se ci si trova meglio e spring lo prenderà in automatico.
  - `src/test/java`: Include un test di base (`contextLoads`) che verifica semplicemente se l'applicazione si avvia correttamente senza errori di configurazione. L'annotazione `@SpringBootTest` sopra i test indica che verrà avviata una versione di prova dell'intera applicazione per verificare che tutti i componenti ("beans") siano caricati correttamente nel contesto di Spring.
- **Gestione con Maven:** Il file `pom.xml` elenca tutte le dipendenze (come `spring-boot-starter-web`) e gestisce il ciclo di vita del progetto (compilazione, test, packaging).

## Implementazione del codice iniziale

Per testare che tutto sia pronto: dopo aver aperto il progetto nell'IDE, si crea la classe `HelloWorldController`. Per renderla funzionale, si utilizzano due annotazioni chiave:

- **@RestController**: Indica a Spring che questa classe gestirà richieste web REST.
- **@GetMapping(path = "/hello")**: Definisce l'endpoint, ne possiamo avere quanti ne vogliamo, sia su più classi che su una singola classe. Quando l'utente visita l'URL `/hello`, il metodo restituisce la stringa "Hello World".

**Esecuzione:** L'app si avvia cliccando sul tasto "Play" della classe principale (`QuickstartApplication`). Di default, il server parte sulla **porta 8080**, e il risultato è visibile nel browser all'indirizzo `http://localhost:8080/hello`.

## HelloWorldController:

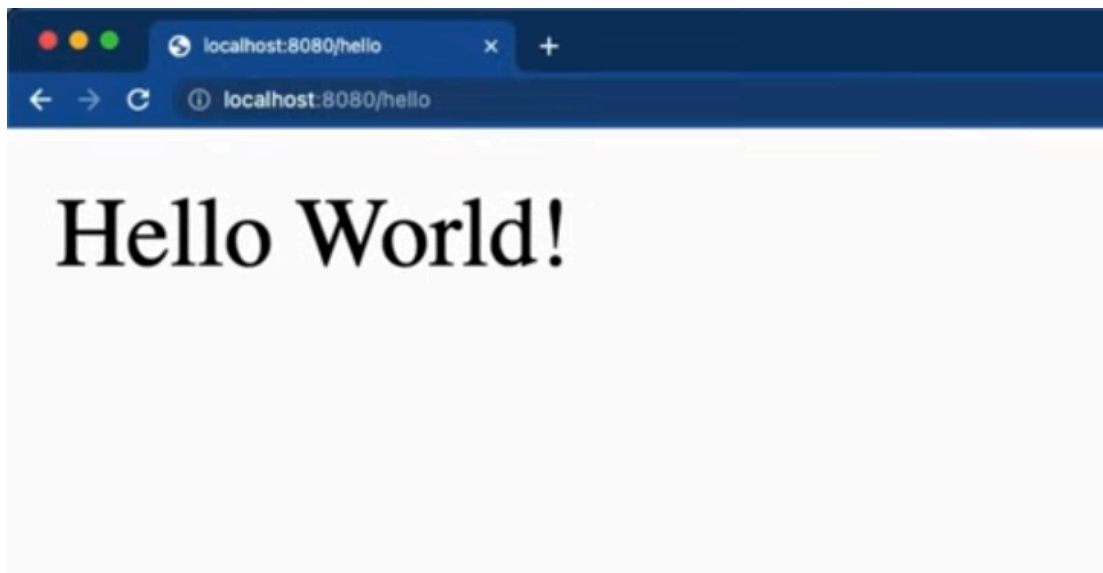
```
package com.devtiro.quickstart;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class HelloWorldController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello World";
    }

    @GetMapping("/ciao")
    public String ciao() {
        return "Ciao Mondo";
    }

    @GetMapping("/hola")
    public String hola() {
        return "Hola Mundo";
    }
}
```

Come viene presentato via browser:



Facendo così abbiamo implementato la nostra prima applicazione SpringBoot.

# Maven

Apache Maven è uno strumento essenziale per la **gestione delle dipendenze e la compilazione (build)** di un progetto software. È una competenza di base essenziale per ogni programmatore. Il suo compito principale è trovare e installare automaticamente le librerie necessarie, ma aiuta anche con test, deployment e altre attività per semplificare il lavoro quotidiano.

## Perché è fondamentale in un progetto?

La funzione principale di Maven è gestire le librerie esterne necessarie al progetto, che andranno elencate nel file `pom.xml`.

- **Automazione:** Maven trova e scarica automaticamente le librerie.
- La sua forza risiede nella gestione delle **dipendenze transitive**: Gestisce le "dipendenze delle dipendenze" di primo, secondo o terzo ordine. Se la tua libreria richiede un'altra risorsa per funzionare, Maven la recupera senza interventi manuali.

## Maven Wrapper (mvnw)

Lo script `mvnw` incluso nei progetti permette di eseguire comandi Maven da riga di comando senza dover installare lo strumento sul proprio computer. Questo garantisce che tutti i membri di un team utilizzino la stessa versione di Maven per lo sviluppo.

- La struttura base è - `mvnw [options] [<goal(s)>] [<phase(s)>]`.

Sebbene Maven sia uno strumento a riga di comando, si integra molto strettamente con ambienti di sviluppo come **IntelliJ IDEA**, che svolge gran parte del lavoro di gestione in background.

## Cicli di Vita e Fasi (Lifecycles & Phases)

Maven organizza il processo di build in tre cicli di vita principali, ognuno dei quali contiene una serie di "obiettivi" (goals) eseguiti in ordine:

- **Clean:** Si occupa della pulizia del progetto. Il comando principale è `clean`, che rimuove la cartella `target` contenente i residui delle compilazioni precedenti.
- **Default:** È il cuore della costruzione del software. Le fasi principali seguono un ordine rigido:
  - `compile`: Qui avviene la compilazione del codice dove si trasforma il codice Java in bytecode.
  - `test`: Esegue gli unit test.
  - `package`: Impacchetta il codice nel formato finale, come un file **JAR** o **WAR**.
  - `verify`: Esegue controlli di qualità e test di integrazione.

**Regola della sequenzialità:** Eseguendo una fase successiva (es. `package`), Maven eseguirà automaticamente tutte le fasi precedenti del ciclo (es. `compile` e `test`).

- **Site:** Genera documentazione tecnica, come JavaDoc e report di progetto.

## La Struttura Standard delle Directory

Maven impone un'organizzazione fissa delle cartelle. Se i file non si trovano nel posto giusto, Maven non sarà in grado di individuarli per la compilazione o i test.

- **src/main/java**: È la cartella che ospita tutto il **codice sorgente Java** dell'applicazione. Al suo interno, il codice è organizzato in pacchetti che riflettono il "Group ID" definito nel progetto (es. `com.devtiro.quickstart`).
- **src/main/resources**: Contiene tutti i file che non sono codice Java ma sono necessari all'app, come i **file di configurazione** (`application.properties` o `application.yml`), file statici o altri dati da includere nel pacchetto finale.
- **src/test/java**: Contiene il codice dei **test unitari e di integrazione**. Per convenzione, la struttura dei pacchetti qui deve rispecchiare esattamente quella di `src/main/java` per facilitare l'associazione tra classi e test relativi.
- **src/test/resources**: Una cartella opzionale ma molto utile per inserire file di configurazione validi **solo durante l'esecuzione dei test**. Spring Boot permette di usare questa cartella per sovrascrivere le impostazioni di produzione (ad esempio, usando un database in memoria per i test invece di quello reale).
- **target**: Questa cartella viene generata automaticamente da Maven durante la build. Contiene tutti i file elaborati: le classi compilate, i report dei test e l'**artefatto finale** (il file `.jar` o `.war`).

### Importanza della Convenzione

Sebbene questa struttura sia un requisito di Maven, **Spring Boot la sfrutta per automatizzare molte operazioni**. Ad esempio, sa automaticamente dove cercare le configurazioni per sovrascriverle nei test se i file sono posizionati correttamente in `src/test/resources`.

## Workflow di Sviluppo Consigliato

Sebbene molti IDE offrano integrazioni grafiche, l'autore suggerisce di utilizzare Maven da riga di comando tramite il **Maven Wrapper (mvnw)** per una maggiore flessibilità, inoltre tende a precisare che non segue la tecnica del developing test driven, quindi questa guida ha un workflow diverso da quello.

1. **Fase di Scrittura Codice:** Nonostante l'IDE dia già di base un aiuto per quanto riguarda i suggerimenti su errori, durante lo sviluppo, è utile eseguire periodicamente `./mvnw compile`. Questo permette di ottenere un feedback rapido dal compilatore per identificare immediatamente eventuali errori di sintassi o configurazione.
2. **Fase di Testing:** Una volta completata una funzionalità, si passa all'esecuzione dei test con `./mvnw clean test`. L'uso del comando `clean` è caldamente raccomandato per assicurarsi che nessun file residuo di compilazioni precedenti influenzi l'esito dei test attuali.
3. **Costruzione dell'Artefatto:** Prima di procedere al commit o al deploy, si genera il file finale con `./mvnw clean package` (o `verify` in caso non volessimo il pacchetto compilato). Questo comando compila il codice, esegue i test e impacchetta l'applicazione in un file **JAR** all'interno della cartella `target`.
4. **Verifica in Stile Produzione:** Una volta generato il JAR, l'autore consiglia di avviarlo manualmente con il comando `java -jar target/[nome_file].jar`. Questo è il modo migliore per testare l'app esattamente come girerà in un ambiente di produzione.

## Metodo Alternativo: Spring Boot Maven Plugin

In alternativa al file JAR, puoi avviare rapidamente l'applicazione usando il **Maven Spring Boot Plugin**, uno strumento integrato nel file `pom.xml` che semplifica l'esecuzione e il confezionamento delle applicazioni Spring Boot.

- **Esecuzione rapida:** Il plugin permette di avviare l'applicazione direttamente dal terminale senza doverla prima compilare e impacchettare manualmente un file JAR. Il comando da utilizzare è: `./mvnw spring-boot:run`.
- **Workflow di Sviluppo:** Rappresenta un'alternativa flessibile all'uso del tasto "Play" dell'IDE o all'esecuzione manuale del JAR tramite il comando `java -jar`. È ideale per testare rapidamente le modifiche durante la scrittura del codice.
- **Gestione del Processo:** Una volta avviata l'applicazione tramite il plugin, questa rimane in esecuzione finché non viene interrotta manualmente. Per fermarla, basta utilizzare la combinazione di tasti **CTRL+C** nel terminale.
- **Packaging (Fat JAR):** Sebbene il video si concentri sull'esecuzione, questo plugin è responsabile della creazione dell'artefatto finale (JAR eseguibile) che include tutte le dipendenze necessarie per far girare l'app in produzione.
- **Integrazione:** Questo plugin lavora in sinergia con il Maven Wrapper (`mvnw`), garantendo che il comando funzioni correttamente su qualsiasi macchina senza richiedere l'installazione globale di Maven.

# Spring Framework & Boot

## Spring Framework vs Spring Boot

Parlando di fatti, Spring Boot è un framework per costruire delle applicazioni Java, dove un Framework è una parte di codice (o layer) costruita sopra il core di un linguaggio di programmazione al fine di poter risolvere problemi comuni, in questa sezione chiariamo il rapporto tra le due tecnologie, definendo Spring Boot come un livello superiore costruito sopra lo Spring Framework.

Ecco le differenze principali:

- **Spring Framework:** È un insieme di codice che risolve problemi comuni (connessione a database, creazione di API REST). Il suo limite è l'elevata configurabilità: essendo estremamente flessibile, richiede allo sviluppatore un grande sforzo manuale per configurare ogni aspetto all'inizio di ogni progetto.
- **Spring Boot:** Nasce per risolvere la complessità di configurazione di Spring. Utilizza le **"convenzioni rispetto alla configurazione"** (Convention over Configuration), fornendo impostazioni predefinite sensate che velocizzano drasticamente lo sviluppo.

In sintesi, usando Spring Boot viene mantenuta tutta la potenza e la flessibilità di Spring Framework, ma con una facilità d'uso molto superiore grazie all'automazione dei processi iniziali.

## Spring App Layers

Qui illustriamo l'architettura a tre livelli (o *layers*) su cui si fondano quasi tutte le applicazioni Spring Boot. Dividere l'app in livelli permette di isolare i problemi, rendendo il codice più facile da scrivere e mantenere.

### 1. Livello di Persistenza (Persistence Layer)

Si trova alla base dell'architettura e si occupa esclusivamente dell'interazione con il database.

- **Entità:** Qui troviamo gli oggetti Java (**Entities**) che rappresentano il dominio dell'applicazione e si mappano direttamente sulle tabelle del database.
- **Pattern:** Vengono utilizzati pattern come **Repository** o **DAO** (Data Access Objects) per esporre le operazioni fondamentali.
- **Funzionalità CRUD:** Lo scopo di questo livello è gestire le operazioni di creazione, lettura, aggiornamento e cancellazione (*Create, Read, Update, Delete*).

### 2. Livello di Servizio (Service Layer)

Rappresenta il cuore pulsante dell'applicazione.

- **Intermediazione:** Impiega le funzionalità fornite dallo strato di persistenza per soddisfare i requisiti del sistema, ospitando di fatto la **logica di business**.

- **Servizi:** Questo avviene attraverso l'utilizzo di classi, identificate come servizi, che implementano specifiche interfacce.
- **Regola d'oro:** Funge da ponte. È fondamentale che il livello di presentazione non parli mai direttamente con quello di persistenza; tutte le chiamate devono passare obbligatoriamente attraverso il livello di servizio.

### 3. Livello di Presentazione (Presentation Layer)

È lo strato più esterno, quello che "espone" i dati all'utente finale.

- **Controller:** In Spring Boot, viene implementato principalmente tramite **Controller** che espongono API REST, ma può supportare anche GraphQL o WebSockets.
- **Flessibilità:** Il vantaggio di questa architettura è che se decidessi di passare da un'API REST a una GraphQL, dovresti modificare solo questo livello, senza toccare la logica nel livello di servizio.

## Modularity

Terminiamo il capitolo dicendo che sappiamo che Spring Framework risolve problemi comuni per noi e che Spring Boot rende più semplice l'utilizzo di Spring Framework. Sappiamo anche che il codice della nostra applicazione è suddiviso in diversi livelli, ovvero il livello di persistenza, il livello di servizio e il livello di presentazione, e che ognuno di questi livelli serve a risolvere un problema specifico.

Parliamo ora del motivo per cui non dobbiamo risolvere ogni problema da soli e di come possiamo sfruttare il vasto ecosistema di dipendenze (o soluzioni pre-confezionate) di Spring, e di come Spring Boot ne renda l'utilizzo molto più semplice.

Supponiamo di voler interagire con un database utilizzando oggetti Java e di scegliere di usare Spring Framework direttamente. Innanzitutto, dovresti includere tutte le dipendenze necessarie nel tuo file `pom.xml`. Dovresti poi creare manualmente diversi bean, tra cui un entity manager per gestire le entità, un transaction manager per gestire le transazioni e numerosi altri bean, e tutto questo prima ancora di scrivere una riga di codice per la logica effettiva dell'applicazione.

Ma se stai usando Spring Boot, ti basterà includere le dipendenze, inserire alcune configurazioni in un file properties su come connetterti al database, e Spring Boot creerà automaticamente tutti i bean necessari, permettendoti di passare direttamente a scrivere la logica della tua applicazione.

Spring Boot crea questi bean utilizzando impostazioni predefinite (**sensible defaults**). Questo non vale solo per il livello di persistenza; lo stesso discorso vale anche per il livello di presentazione o per qualsiasi altro livello. Se volessi creare un'API REST, di solito è sufficiente includere le dipendenze Spring Web e il gioco è fatto. Ora che abbiamo trattato l'approccio generale, parliamo di come Spring raggiunga concretamente questo risultato: parliamo dell'Iniezione delle Dipendenze (**Dependency Injection**).

# Dependency Injection

## Inversion of Control (IoC)

Procediamo con lo spiegare il principio filosofico alla base di Spring, che trasforma il modo in cui le classi interagiscono tra loro.

### Il Problema: Accoppiamento Forte

Tradizionalmente, se una classe (chiamiamola "Verde" per semplificare l'esempio) ha bisogno di altre classi (Blu, Gialla e Rossa) per funzionare, le crea internamente usando la parola chiave `new`.

- **Svantaggio:** Questo crea un legame rigido. Se devi sostituire la Classe Blu con una versione diversa, sei costretto a modificare il codice sorgente della Classe Verde.

### La Soluzione: Inversione del Controllo

Con l'IoC, la responsabilità di creare e gestire le dipendenze viene tolta alla classe stessa e affidata a un'entità esterna di livello superiore: il **framework** (Spring).

- **Come funziona:** La Classe Verde non crea più i suoi oggetti, ma dichiara semplicemente di quali **interfacce** ha bisogno.
- **L'intervento di Spring:** Il framework agisce come un contenitore esterno che crea le classi concrete e le "inietta" dove sono state dichiarate le interfacce.

### Vantaggi della Modularità

Questo approccio rende l'applicazione estremamente flessibile. Se vuoi cambiare un componente (ad esempio, sostituire una classe "Alpha" con una nuova implementazione), ti basta farlo all'interno del framework. La classe che riceve la dipendenza non deve essere minimamente modificata, poiché continua a vedere solo l'interfaccia.

### IoC vs. Dependency Injection

Sebbene i termini siano spesso usati come sinonimi, il video chiarisce la distinzione tecnica:

- **Inversion of Control (IoC):** È il concetto generale di spostare il controllo delle dipendenze all'esterno della classe.
- **Dependency Injection (DI):** È il pattern specifico (termine coniato da Martin Fowler) che Spring utilizza per implementare concretamente l'IoC.

Questa parte è fondamentale perché mostra come Spring sia così potente nel gestire sistemi complessi.

## Introducing Beans

### Definizione di "Bean"

Qui si entra nel vivo della pratica, spiegando come Spring gestisce concretamente gli oggetti.

In Spring, il termine **Bean** identifica semplicemente una **classe concreta** (un oggetto Java) che viene creata e gestita interamente dal framework anziché dallo sviluppatore. Invece di usare la parola chiave `new` per istanziare manualmente un oggetto, si delega questo compito a Spring, rendendo l'applicazione più modulare e meno "accoppiata".

### Il concetto di "Application Context"

Tutti i Beans creati vengono inseriti in quello che il video definisce come un **"pool"** o contenitore, chiamato **Spring Application Context**. Quando una parte del software ha bisogno di una dipendenza (ad esempio, un controller che necessita di un servizio), Spring "pesca" il Bean corrispondente da questo pool e lo inietta automaticamente.

### Metodi per dichiarare i Beans

Esistono due modi principali per comunicare a Spring quali classi trasformare in Beans:

- **Dichiarazione Esplicita (@Configuration + @Bean):**
  - Si crea una classe annotata con **@Configuration**, che funge da "mappa" per il framework.
  - All'interno, si definiscono metodi annotati con **@Bean** che restituiscono un'istanza della classe desiderata.
  - Questo approccio è utile quando si ha bisogno di configurare manualmente l'oggetto prima di renderlo disponibile.
- **Dichiarazione Automatica (@Component e @Service):**
  - Annotando una classe direttamente con **@Component**, si dice a Spring: "Questa classe è un Bean, inseriscila nel contesto".
  - **@Service** è un "amico" di **@Component**: fa la stessa cosa ma è più descrittivo per le classi che risiedono nel **Service Layer** (livello di logica di business).

### Vantaggio Pratico: Swap delle Dipendenze

Il video dimostra la potenza dei Beans con l'esempio delle classi "Printer". Se hai bisogno di cambiare il comportamento di una dipendenza (ad esempio, passare da una stampante che scrive in inglese a una in spagnolo), non devi modificare il codice della classe che la usa. Ti basta cambiare la dichiarazione del Bean nella configurazione o spostare l'annotazione **@Component** sulla nuova classe concreta; Spring si occuperà di aggiornare l'iniezione in automatico.

Questa sezione è il ponte tra la teoria (IoC) e la pratica dello sviluppo.

Nel video viene mostrato come una applicazione passa dall'essere una tipica applicazione non modulare con dipendenze ad una che utilizza dei semplici beans, il primo esempio infatti mostra come si passa da:

```
package com.devtiro.maven;

import com.devtiro.maven.services.ColourPrinter;
import com.devtiro.maven.services.impl.ColourPrinterImpl;
import lombok.extern.java.Log;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
@Log
public class ColoursApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(ColoursApplication.class, args);
    }
    @Override
    public void run(final String... args) {
        final ColourPrinter colourPrinter = new ColourPrinterImpl();
        Log.info(colourPrinter.print());
    }
}
```

A questo

```
package com.devtiro.maven;

import com.devtiro.maven.services.ColourPrinter;
import lombok.extern.java.Log;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
@Log
public class ColoursApplication implements CommandLineRunner {

    private ColourPrinter colourPrinter;

    public ColoursApplication(ColourPrinter colourPrinter) {
        this.colourPrinter = colourPrinter;
    }

    public static void main(String[] args) {
        SpringApplication.run(ColoursApplication.class, args);
    }
    @Override
    public void run(final String... args) {
        Log.info(colourPrinter.print());
    }
}
```

E da questo

```
package com.devtiro.maven.services.impl;

import com.devtiro.maven.services.BluePrinter;
import com.devtiro.maven.services.ColourPrinter;
import com.devtiro.maven.services.GreenPrinter;
import com.devtiro.maven.services.RedPrinter;

public class ColourPrinterImpl implements ColourPrinter {
    private RedPrinter redPrinter;
    private BluePrinter bluePrinter;
    private GreenPrinter greenPrinter;

    public ColourPrinterImpl() {
        this.redPrinter = new EnglishRedPrinter();
        this.bluePrinter = new EnglishBluePrinter();
        this.greenPrinter = new EnglishGreenPrinter();
    }
    @Override
    public String print() {
        return String.join(" ", redPrinter.print(), bluePrinter.print(),
greenPrinter.print());
    }
}
```

A questo

```
package com.devtiro.maven.services.impl;

import com.devtiro.maven.services.BluePrinter;
import com.devtiro.maven.services.ColourPrinter;
import com.devtiro.maven.services.GreenPrinter;
import com.devtiro.maven.services.RedPrinter;

public class ColourPrinterImpl implements ColourPrinter {
    private RedPrinter redPrinter;
    private BluePrinter bluePrinter;
    private GreenPrinter greenPrinter;

    public ColourPrinterImpl(
        RedPrinter redPrinter, BluePrinter bluePrinter, GreenPrinter
greenPrinter) {
        this.redPrinter = redPrinter;
        this.bluePrinter = bluePrinter;
        this.greenPrinter = greenPrinter;
    }
    @Override
    public String print() {
        return String.join(" ", redPrinter.print(), bluePrinter.print(),
greenPrinter.print());
    }
}
```

Aggiungendo la classe config dove possono essere poi modificate le varie classi che ci servono da richiamare.

```
package com.devtiro.maven.config;

import com.devtiro.maven.services.BluePrinter;
import com.devtiro.maven.services.ColourPrinter;
import com.devtiro.maven.services.GreenPrinter;
import com.devtiro.maven.services.RedPrinter;
import com.devtiro.maven.services.impl.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class PrinterConfig {

    @Bean
    public BluePrinter bluePrinter() {
        return new SpanishBluePrinter();
    }

    @Bean
    public RedPrinter redPrinter() {
        return new SpanishRedPrinter();
    }

    @Bean
    public GreenPrinter greenPrinter() {
        return new SpanishGreenPrinter();
    }

    @Bean
    public ColourPrinter colourPrinter(
        BluePrinter bluePrinter, RedPrinter redPrinter, GreenPrinter
greenPrinter) {
        return new ColourPrinterImpl(redPrinter, bluePrinter, greenPrinter);
    }
}
```

Tutto ciò lasciando invariate le varie classi come EnglishPrinter, ColourPrinter, BluePrinter ecc...

```
package com.devtiro.maven.services;

public interface BluePrinter {
    String print();
}
```

```
package com.devtiro.maven.services;

public interface ColourPrinter {
    String print();
}
```

```
package com.devtiro.maven.services.impl;

import com.devtiro.maven.services.BluePrinter;

public class EnglishBluePrinter implements BluePrinter {

    @Override
    public String print() {
        return "blue";
    }
}
```

## @Component and Friends

Con la premessa precedente adesso vediamo come automatizzare la creazione dei Beans senza doverli definire manualmente uno per uno nelle classi di configurazione.

### L'annotazione @Component

È l'annotazione fondamentale per la dichiarazione automatica dei Beans.

- **Registrazione automatica:** Aggiungendo `@Component` sopra una classe, si comunica a Spring: "Questa classe è un Bean, creane un'istanza e inseriscila nell'Application Context".
- **Gestione dipendenze:** Indica a Spring di risolvere e iniettare automaticamente tutte le dipendenze dichiarate nel costruttore di quella classe.

### Gli "Amici" di @Component (Stereotipi)

Il video introduce il concetto di "stereotipi", ovvero annotazioni specializzate che funzionano tecnicamente come `@Component` ma aggiungono un significato semantico e architettonico.

- **@Service:** È il principale "amico" presentato in questa sezione.
  - **Funzionamento:** Fa esattamente la stessa cosa di `@Component` (registra la classe come Bean).
  - **Scopo:** Viene utilizzata per rendere il codice più descrittivo, identificando chiaramente le classi che appartengono al **Service Layer** (il livello della logica di business).

### Esempio di flessibilità

Il video dimostra come queste annotazioni facilitino lo scambio di implementazioni. Ad esempio, per passare da una stampante in inglese a una in spagnolo, basta rimuovere `@Component` dalla prima e aggiungerla (o usare `@Service`) sulla seconda. Spring aggiornerà l'iniezione ovunque sia necessario senza richiedere modifiche al codice che utilizza quelle dipendenze.

Anche qui vi sarebbe un esempio all'interno dell'IDE, ma banalmente quello che viene mostrato è la cancellazione della classe `PrinterConfig`, ed all'interno delle classi `ColourPrinterImpl`, `SpanishBluePrinter` ecc... viene aggiunto il `@Component` oppure `@Service`.

```
@Component
public class ColourPrinterImpl implements ColourPrinter {...}
```

```
@Component
public class EnglishRedPrinter implements RedPrinter {...}
```

```
@Service
public class SpanishBluePrinter implements BluePrinter {...}
```

## Component Scanning

*Il processo automatico con cui Spring individua e registra i componenti dell'applicazione all'avvio.*

### Definizione e Avvio

Il **Component Scanning** è un processo che si attiva nel momento in cui l'applicazione viene lanciata. Spring scandisce il progetto alla ricerca di classi annotate come Beans (tramite `@Component`, `@Service` o `@Bean` nelle classi di configurazione) e identifica dove queste dipendenze sono richieste.

### Il Processo di Registrazione

Durante la scansione, Spring esegue due passaggi principali:

- **Individuazione:** Se trova una classe con `@Component` che non ha dipendenze, ne crea un'istanza e la inserisce nello **Spring Application Context** (il "pool" dei Beans).
- **Risoluzione delle Dipendenze:** Se una classe richiede altre dipendenze (ad esempio tramite un costruttore), Spring cerca nel pool i Beans necessari e li "inietta". Questo processo è chiamato anche **Auto-wiring**.

### Gerarchia e Punto di Partenza

La scansione non avviene su tutto il computer, ma parte da un punto preciso della gerarchia del progetto e procede verso il basso, includendo tutti i sotto-pacchetti.

- **Convenzione:** In Spring Boot, questo punto di partenza è definito dalla posizione della classe principale annotata con `@SpringBootApplication`.
- **Limiti:** Se una classe viene spostata in un pacchetto che si trova "sopra" o al di fuori della cartella che contiene la classe principale, Spring non la troverà e l'applicazione fallirà perché non riuscirà a creare i Beans necessari.

### Il Ruolo di `@SpringBootApplication`

L'annotazione `@SpringBootApplication` è un alias che include al suo interno `@ComponentScan`. Questo significa che, per impostazione predefinita, Spring Boot configura automaticamente la scansione partendo dal pacchetto della classe `main`, eliminando la necessità di configurazioni manuali complicate che erano tipiche del framework Spring tradizionale.

## @SpringBootApplication Annotation

*Il "cuore" e punto di ingresso di ogni applicazione Spring Boot.*

Una "Super Annotazione" (Alias)

L'annotazione `@SpringBootApplication` non è una singola istruzione, ma un alias che raggruppa tre annotazioni principali che automatizzano la configurazione del framework:

1. **@Configuration**: Identifica la classe come una fonte di definizioni di **Beans**. Indica a Spring che all'interno di questa classe (o di quelle scansionate) può trovare metodi per configurare i componenti del sistema.
2. **@ComponentScan**: Attiva la ricerca automatica di componenti (come `@Component` o `@Service`). Il punto di forza è che la scansione parte dal pacchetto in cui si trova la classe principale e procede verso tutti i sotto-pacchetti, risparmiando allo sviluppatore di dover configurare manualmente i percorsi di ricerca.
3. **@EnableAutoConfiguration**: È la base della "magia" di Spring Boot. Questa annotazione istruisce il framework a guardare le librerie presenti nel progetto (i "starters") e a creare automaticamente i Beans necessari con impostazioni predefinite ragionevoli (*sensible defaults*). Ad esempio, se trova la libreria per il web, configura automaticamente un server Tomcat.

Il Punto di Ingresso (Entry Point)

Oltre alla configurazione, questa annotazione identifica la classe che contiene il metodo `main`, ovvero il punto esatto da cui l'applicazione viene avviata. Senza di essa, Spring Boot non saprebbe da dove iniziare la scansione dei componenti o come attivare l'autoconfigurazione.

# Autoconfiguration

## Cos'è l'Autoconfigurazione?

Adesso descriviamo il meccanismo "magico" con cui Spring Boot configura automaticamente l'applicazione in base alle librerie incluse nel progetto.

L'autoconfigurazione è il processo che Spring Boot avvia al momento del lancio per fornire **"sensible defaults"** (impostazioni predefinite ragionevoli) e creare automaticamente le dipendenze necessarie. Invece di costringere lo sviluppatore a configurare manualmente ogni componente, Spring Boot capisce ciò di cui hai bisogno osservando il tuo progetto.

## Il Ruolo degli "Starters"

L'autoconfigurazione lavora a stretto contatto con i **Spring Boot Starters**, che sono collezioni di dipendenze raggruppate per risolvere un problema specifico.

- **Esempio:** Includendo `spring-boot-starter-web`, carichi non solo le librerie web, ma anche un server Tomcat incorporato e il supporto per JSON.

## Come Funziona il Meccanismo

Il framework analizza il **classpath** del progetto. Se rileva la presenza di determinate classi o librerie, attiva delle classi di configurazione specifiche. Questo avviene tramite annotazioni condizionali:

- **@ConditionalOnClass:** Questa annotazione dice a Spring di istanziare un componente solo se una specifica classe è presente nel progetto. Ad esempio, se trova la classe del `DispatcherServlet`, configura automaticamente l'ambiente web.
- **@ConditionalOnMissingBean:** È fondamentale per la flessibilità. Dice a Spring di creare un Bean predefinito **solo se lo sviluppatore non ne ha già definito uno proprio**. Questo ti permette di sovrascrivere facilmente le impostazioni automatiche quando necessario.
- **@ConditionalOnBean:** A differenza della scorsa citata, questa **agisce in presenza di un bean**. In pratica, dice a Spring: "Esegui questa configurazione solo se un determinato Bean è già disponibile nel contesto".
  - Ad esempio, se nel contesto è presente un `DataSource`, Spring Boot potrebbe attivare automaticamente la configurazione di un `JdbcTemplate`. Se il `DataSource` non c'è, quella parte di configurazione viene saltata.

## Esempio Pratico: DispatcherServlet

Nel video viene mostrato il `DispatcherServletAutoConfiguration`. In un progetto Spring tradizionale, dovresti configurarlo manualmente; in Spring Boot, se la dipendenza web è presente, il framework crea e configura il Bean per te senza che tu debba scrivere una sola riga di codice.

Questa sezione spiega bene perché Spring Boot è così veloce per iniziare nuovi progetti ed è solo per dare una conoscenza di base di come funziona Spring, ciò non è strettamente necessario al fine di poterlo utilizzare.

Come viene citato nel video, non serve conoscere come funziona ogni parte di una macchina per poterla guidare.

# Configuration

## Configuration files

In questa sezione approfondiremo come personalizzare il comportamento di un'applicazione Spring Boot sovrascrivendo le impostazioni predefinite tramite file dedicati senza toccare il codice Java.

### Scopo dei file di configurazione

Sebbene Spring Boot fornisca "sensible defaults", spesso è necessario modificarle. L'esempio classico nel video è il cambio della porta del server Tomcat: di default è la **8080**, ma può essere spostata, ad esempio, sulla **8181** tramite file di configurazione.

Da notare che tutte le chiavi di configurazione, impostazioni ecc..., sono reperibili facilmente presso il sito web di Spring-Boot nella sezione [Common Application Properties](#).

### Formati supportati: Properties e YAML

Spring Boot supporta due formati principali per questi file, entrambi situati solitamente in **src/main/resources**:

- **application.properties**: Utilizza il formato **chiave=valore** (es. **server.port=8181**). È il formato tradizionale e più semplice.
- **application.yml (YAML)**: Utilizza l'indentazione per strutturare i dati, evitando la ripetizione delle chiavi:

```
server:
  port: 8282
```

Molti sviluppatori lo preferiscono per la sua leggibilità nei progetti complessi.

### Documentazione delle proprietà comuni

Esistono centinaia di proprietà configurabili. Il video suggerisce di consultare la pagina **"Common Application Properties"** nella documentazione ufficiale di Spring, dove sono elencate chiavi per ogni ambito: dai log (**debug=true**) alla gestione del database, fino alle impostazioni del server.

### Configurazione per i Test

Un concetto cruciale è l'uso della cartella **src/test/resources**. Se inserisci un file **application.properties** in questa posizione, i valori lì definiti verranno preferiti rispetto a quelli di produzione solo durante l'esecuzione dei test.

- **Esempio pratico**: In produzione potresti avere la configurazione per un database reale (come PostgreSQL), mentre in **src/test/resources** configuri un database in memoria (come H2) per rendere i test veloci e isolati.

## Environment Variables

Qui verrà illustrato come le variabili d'ambiente rappresentino uno dei metodi più potenti e flessibili per configurare un'applicazione Spring Boot, specialmente in scenari di distribuzione moderna come Docker o il cloud.

### Importanza delle Variabili d'Ambiente

Mentre i file `application.properties` sono utili per i valori predefiniti, le variabili d'ambiente sono fondamentali per configurare l'applicazione all'esterno del codice sorgente. Questo è essenziale quando si eseguono applicazioni in container (come Docker), dove è prassi comune passare configurazioni (come credenziali del database o porte) tramite l'ambiente.

### Regole di Conversione (Naming Convention)

Spring Boot utilizza una logica di **"fuzzy matching"** per mappare le variabili d'ambiente alle chiavi di configurazione del framework. Per convertire una proprietà standard in una variabile d'ambiente, segui queste regole:

- **Tutto in maiuscolo:** La chiave deve essere scritta interamente in lettere maiuscole.
- **Sostituzione dei delimitatori:** Ogni punto (.) o trattino (-) deve essere sostituito con un carattere di sottolineatura (\_).
- **Esempio pratico:** La proprietà `server.port` diventa la variabile d'ambiente `SERVER_PORT`.

### Modalità di Utilizzo

Il video mostra diversi modi per applicare queste variabili:

- **Tramite IDE:** In IntelliJ è possibile aggiungere variabili d'ambiente direttamente nella configurazione di avvio, nella versione attuale che sto utilizzando di IntelliJ, questa opzione si trova dentro la sezione Run/Debug Configuration del Progetto (I tre puntini in alto a destra oppure premi Shift 2 volte e ricerchi "edit configurations" per accedere a quel menù), e si può attivare tramite shortcut Alt+E.
- **Riga di comando (Inline):** È possibile passare la variabile subito prima del comando di avvio, ad esempio: `SERVER_PORT=8282 ./mvnw spring-boot:run`.
- **Esportazione globale:** Si può esportare la variabile nel terminale (`export SERVER_PORT=8383`) affinché ogni esecuzione successiva (sia tramite Maven che tramite file JAR diretto) utilizzi quel valore.

### Ordine di Priorità

Un aspetto cruciale è che le variabili d'ambiente hanno la precedenza sui valori definiti nei file `application.properties` o `application.yml` situati nel progetto. Questo permette di sovrascrivere le impostazioni locali senza modificare il pacchetto applicativo.

## Configuration Properties

*Utili per gestire la configurazione personalizzata in modo strutturato, mappando gruppi di chiavi direttamente su oggetti Java "fortemente tipizzati".*

Cos'è la Configuration Properties?

A differenza delle proprietà singole, le **Configuration Properties** permettono di raggruppare impostazioni correlate in un unico oggetto Java. Questo approccio offre due vantaggi principali:

- **Modularità:** Tutta la configurazione per una specifica funzione (es. le impostazioni di una "Pizza") è centralizzata in una classe.
- **Integrazione:** Questi oggetti sfruttano tutti i canali di configurazione di Spring Boot (file `.properties`, `.yaml`, variabili d'ambiente) esattamente come le proprietà di sistema.

L'annotazione `@ConfigurationProperties` e il prefix

Il meccanismo si attiva tramite l'annotazione `@ConfigurationProperties`. L'elemento cruciale è l'argomento `prefix`, che indica a Spring la "radice" delle chiavi da cercare.

- Se definisci un prefisso `pizza`, Spring cercherà nel file di configurazione tutte le chiavi che iniziano con `pizza.*` (come `pizza.sauce`, `pizza.topping`, ecc.) e proverà a iniettarne i valori nei campi corrispondenti della classe.

Requisiti e Struttura della Classe

Perché il mapping funzioni, la classe deve essere costruita seguendo regole precise:

- **Corrispondenza dei Nomi:** I nomi delle variabili di istanza devono coincidere con i nomi delle chiavi (es. variabile `sauce` per la chiave `pizza.source`).
- **Getter, Setter e Costruttori:** La classe necessita di metodi getter/setter e di un costruttore senza argomenti (no-args constructor) per permettere a Spring di istanziarla e popolarla.
- **Lombok:** Nel video, l'autore utilizza le annotazioni di **Lombok** (come `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`) per generare automaticamente questo codice ripetitivo, mantenendo la classe pulita.

Abilitazione e Iniezione

Per rendere operativa la classe, occorre annotarla anche con `@Configuration`. Questo comunica a Spring di scansionare la classe all'avvio e registrarla come un **Bean** nell'Application Context. Una volta registrata, la classe di configurazione può essere iniettata tramite costruttore in qualsiasi altro componente (come un servizio o un controller), rendendo i dati configurati pronti all'uso.

## Esempio Pratico: PizzaConfig

L'esempio del video mostra una classe `PizzaConfig` con i campi `source`, `topping` e `crust`. Definendo i valori nel file `application.properties` (es. `pizza.sauce=tomato`), Spring popola l'oggetto all'avvio. Se un valore viene poi passato tramite variabile d'ambiente (es. `PIZZA_SOURCE=barbecue`), questo sovrascriverà il valore del file, garantendo massima flessibilità senza toccare il codice.

Questa tecnica è essenziale per scrivere codice pulito e facilmente configurabile in ambienti diversi.

Di fatto come da esempio si passa da una configurazione statica come questa:

```
package com.devtiro.configuration;

import com.devtiro.configuration.config.PizzaConfig;
import lombok.Builder;
import lombok.extern.java.Log;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@Log
public class PizzaApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(PizzaApplication.class, args);
    }

    @Override
    public void run(final String... args) {
        final PizzaConfig pizzaConfig = new PizzaConfig(
            "tomato", "mozzarella", "thin"
        );

        Log.info(
            String.format("I want a %s crust pizza, with %s and %s",
                pizzaConfig.getCrust(),
                pizzaConfig.getTopping(),
                pizzaConfig.getSauce()
            ));
    }
}
```

```
package com.devtiro.configuration.config;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Configuration;

@AllArgsConstructor
@NoArgsConstructor
@Data
public class PizzaConfig {
    private String sauce;
    private String topping;
    private String crust;
}
```

Ad una dinamica e flessibile dove PizzaConfig è un bean e sauce, toppings e crust possono essere configurate tramite application.properties, variabili di ambiente oppure via console:

```
package com.devtiro.configuration;

import com.devtiro.configuration.config.PizzaConfig;
import lombok.Builder;
import lombok.extern.java.Log;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@Log
public class PizzaApplication implements CommandLineRunner {

    private PizzaConfig pizzaConfig;

    public PizzaApplication(PizzaConfig pizzaConfig) {
        this.pizzaConfig = pizzaConfig;
    }

    public static void main(String[] args) {
        SpringApplication.run(PizzaApplication.class, args);
    }

    @Override
    public void run(final String... args) {
        Log.info(
            String.format("I want a %s crust pizza, with %s and %s",
                pizzaConfig.getCrust(),
                pizzaConfig.getTopping(),
                pizzaConfig.getSauce()
            ));
    }
}
```

```

package com.devtiro.configuration.config;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Configuration;

@Configuration
@ConfigurationProperties(prefix = "pizza")
@AllArgsConstructor
@NoArgsConstructor
@Data
public class PizzaConfig {
    private String sauce;
    private String topping;
    private String crust;
}

```

Dove il application.properties ha questa struttura:

```

pizza.sauce=bbq
pizza.topping=chicken
pizza.crust=stuffed

```

Adesso possiamo passare alla configurazione di un database.

# Databases Part 1 - Basics

Va tenuto conto che questa sezione serve come infarinatura, di fatto andando avanti impiegheremo un database definitivo, quindi non panicare (come ho fatto io inizialmente) per le varie tipologie di database che verranno proposte successivamente.

## Database Layers

Cominciamo con il descrivere i diversi livelli tecnologici che permettono a un'applicazione Java di comunicare con un database, partendo dal livello più vicino alla macchina fino a quello più astratto.

### Database Driver

Alla base di tutto c'è il **driver del database**. Ogni database relazionale (come PostgreSQL o MySQL) fornisce un driver Java specifico che agisce come traduttore, permettendo al codice Java di inviare comandi al database.

### JDBC (Java Database Connectivity)

Appena sopra il driver troviamo **JDBC**, un'API di basso livello.

- **Caratteristiche:** Gli sviluppatori interagiscono con il database scrivendo query SQL esplicite.
- **Vantaggi/Svantaggi:** Offre un controllo granulare sulle query, permettendo di ottimizzarle al massimo, ma richiede che lo sviluppatore gestisca manualmente la mappatura dei dati tra le tabelle SQL e gli oggetti Java, un lavoro spesso ripetitivo e noioso.

### Spring JDBC

Questo livello è un'estensione di Spring che semplifica l'uso di JDBC tramite strumenti come il **JdbcTemplate**. Pur continuando a usare SQL, Spring JDBC elimina gran parte del codice "boilerplate" (di contorno) necessario per gestire connessioni ed eccezioni.

### JPA (Java Persistence API) e Hibernate

JPA è un'API di alto livello che introduce il concetto di **ORM (Object Relational Mapper)**.

- **Funzionamento:** Invece di scrivere SQL, lo sviluppatore interagisce con il database usando direttamente oggetti Java. JPA si occupa di generare l'SQL necessario e di mappare i risultati sugli oggetti.
- **Specific vs Implementazione:** Tecnicamente, JPA è solo una "specific" (un insieme di regole); **Hibernate** è l'implementazione concreta più comune utilizzata nell'ecosistema Spring.

## Spring Data JPA

Si trova in cima alla piramide e aggiunge ulteriore automazione sopra JPA. Introduce i **Repository**, che permettono di gestire le operazioni sui dati con pochissimo codice, automatizzando ulteriormente la creazione delle query.

Questo schema a strati è fondamentale per capire come Spring possa gestire i dati in modo così efficiente.

## Connect to a H2 Database

In questa sezione spieghiamo come utilizzare H2, il database in memoria di riferimento nell'ecosistema Spring, per testare rapidamente le applicazioni senza configurazioni esterne.

### Caratteristiche di H2

H2 è un database "in-memory", il che significa che i dati risiedono nella RAM e vengono persi al riavvio dell'applicazione. È perfetto per demo, prototipazione e test automatizzati poiché non richiede l'installazione di un server database fisico sul computer.

### Configurazione e Dipendenze

Per iniziare, durante la creazione del progetto su Spring Initializr, è necessario selezionare le seguenti dipendenze:

- **H2 Database:** Il motore del database stesso.
- **JDBC API:** Per permettere la connettività Java.
- **Lombok:** Utilizzato nel video per semplificare la creazione di logger e costruttori.

### La "Magia" dell'Autoconfigurazione

Il punto di forza di Spring Boot con H2 è l'automazione. Se il framework rileva la libreria H2 nel **classpath**, configura automaticamente i Bean necessari per la connessione senza che lo sviluppatore debba scrivere nulla nel file `application.properties`. Spring Boot assegna valori predefiniti per URL, username e password in modo trasparente.

### Configurazione Manuale (Opzionale)

Sebbene automatica, è possibile definire esplicitamente i parametri nel file `application.properties` per avere maggiore controllo:

- `spring.datasource.url=jdbc:h2:mem:testdb` (indica che il database è in memoria e si chiama "testdb").
- `spring.datasource.driverClassName=org.h2.Driver`.
- `spring.datasource.username=sa` (utente predefinito "System Administrator").
- `spring.datasource.password=password`.

### Verifica della Connessione

Nel video, l'autore dimostra la connessione iniettando un oggetto **DataSource** nel costruttore della classe principale ed eseguendo una query di test ("SELECT 1") tramite **JdbcTemplate**. I log mostrano l'avvio del pool di connessioni (HikariPool) e la conferma che l'applicazione è riuscita a interrogare correttamente il database in memoria.

Il codice mostrato in video per il testing della connessione ad un database H2 è:

```
package com.devtiro.database;

import lombok.extern.java.Log;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.jdbc.core.JdbcTemplate;

import javax.sql.DataSource;

@SpringBootApplication
@Log
public class DatabaseApplication implements CommandLineRunner {

    private final DataSource dataSource;

    public DatabaseApplication(final DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public static void main(String[] args) {
        SpringApplication.run(DatabaseApplication.class, args);
    }

    @Override
    public void run(final String... args) {
        Log.info("Datasource: " + dataSource.toString());
        final JdbcTemplate restTemplate = new JdbcTemplate(dataSource);
        restTemplate.execute("select 1");
    }
}
```

Dove, come anticipato precedentemente, l'application.properties è configurato come:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
```

## Connect to a PostgreSQL Database

*Il passaggio dall'uso di un database di test in memoria a un database relazionale reale e persistente.*

### Dipendenze necessarie

Per connettersi a PostgreSQL, non basta più la configurazione automatica di H2. Tramite Spring Initializr, è necessario aggiungere:

- **PostgreSQL Driver:** Il driver specifico per permettere a Java di comunicare con PostgreSQL.
- **JDBC API:** Per gestire la connettività e le query.
- **Lombok:** Utilizzato per generare automaticamente i logger (`@Log`) e gestire l'iniezione tramite costruttore.

### Configurazione del Database (Docker Compose)

A differenza di H2, PostgreSQL deve essere un processo esterno attivo. Nel video viene utilizzato un file **Docker Compose** per avviare rapidamente un'istanza del database con queste specifiche:

- **Immagine:** `postgres`.
- **Porta:** `5432`.
- **Password:** Una password personalizzata (es. `ChangeMeInProd!`) definita come variabile d'ambiente nel file Docker.

### Configurazione in `application.properties`

Per istruire Spring Boot su come trovare il database, occorre popolare il file di configurazione con parametri espliciti:

- `spring.datasource.url`: L'indirizzo JDBC (es. `jdbc:postgresql://localhost:5432/postgres`).
- `spring.datasource.username`: L'utente predefinito (solitamente `postgres`).
- `spring.datasource.password`: La password impostata nel container Docker.
- `spring.datasource.driver-class-name`: Il nome della classe del driver (`org.postgresql.Driver`).

### Gestione degli errori e Avvio

Se l'applicazione viene avviata senza un database funzionante o senza configurazione, fallirà immediatamente con errori come "fail to determine a suitable driver class" o "connection refused". Una volta avviato il database tramite comando:

- `"docker-compose up"`.

e configurato Spring Boot, i log mostreranno l'attivazione di **HikariPool** e la connessione riuscita all'istanza PostgreSQL.

Ovviamente ciò con Docker installato (Ottime guide per installarlo si trovano sul sito di [IntelliJ](#) o direttamente dal [Docker Docs](#), per windows consiglio di installarlo direttamente dallo store e poi eseguire il comando da console `wsl --update`).

### Verifica della connessione

Per confermare che tutto funzioni, il video mostra come iniettare il **DataSource** ed eseguire una query di test minima (`SELECT 1`) tramite il **JdbcTemplate**. Se la query viene eseguita senza eccezioni, la comunicazione è stabilita correttamente.

Il codice mostrato per la connessione al database di PostgreSQL è:

```
package com.devtiro.database;

import lombok.extern.java.Log;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.jdbc.core.JdbcTemplate;

import javax.sql.DataSource;

@SpringBootApplication
@Log
public class DatabaseApplication implements CommandLineRunner {

    private final DataSource dataSource;

    public DatabaseApplication(final DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public static void main(String[] args) {
        SpringApplication.run(DatabaseApplication.class, args);
    }

    @Override
    public void run(final String... args) {
        Log.info("Datasource: " + dataSource.toString());
        final JdbcTemplate restTemplate = new JdbcTemplate(dataSource);
        restTemplate.execute("select 1");
    }
}
```

Il docker-compose.yml:

```
version: '3.1'

services:

  db:
    image: postgres
    ports:
      - "5432:5432"
    restart: always
    environment:
      POSTGRES_PASSWORD: changemeinprod!
```

Ed infine l'application.properties come descritto sopra:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=changemeinprod!
spring.datasource.driver-class-name=org.postgresql.Driver
```

## Initialise DB Schema

Infine vediamo come automatizzare la creazione delle tabelle e l'inserimento dei dati iniziali nel database all'avvio dell'applicazione.

I file di inizializzazione SQL

Spring Boot cerca per convenzione due file specifici all'interno della cartella

**src/main/resources:**

- **schema.sql**: Viene utilizzato per definire la struttura del database. Qui si inseriscono i comandi DDL (*Data Definition Language*), come **CREATE TABLE** o **DROP SEQUENCE**, per preparare l'ambiente.

```
DROP TABLE IF EXISTS "widgets";

DROP SEQUENCE IF EXISTS widgets_id_seq;
CREATE SEQUENCE widgets_id_seq INCREMENT 1 MINVALUE 1 MAXVALUE
9223372036854775807 CACHE 1;

CREATE TABLE "widgets" (
  "id" bigint DEFAULT nextval('widgets_id_seq') NOT NULL,
  "name" text,
  "purpose" text,
  CONSTRAINT "widgets_pkey" PRIMARY KEY ("id")
);
```

- **data.sql**: Viene utilizzato per popolare le tabelle create. Qui si inseriscono i comandi DML (*Data Manipulation Language*), come **INSERT INTO**, per caricare dati di test o configurazioni iniziali.

```
INSERT INTO widgets (id, name, purpose) VALUES
(1, 'Widget A', 'Used for testing purposes.'),
(2, 'Widget B', 'Designed for entertainment.'),
(3, 'Widget C', 'Enhances productivity.'),
(4, 'Widget D', 'Perfect for outdoor activities.'),
(5, 'Widget E', 'Improves overall well-being.');
```

## Configurazione in application.properties

Affinché Spring Boot esegua effettivamente questi file, è necessario abilitare esplicitamente la modalità di inizializzazione nel file di configurazione:

- **Proprietà:** `spring.sql.init.mode=always`.
- **Significato:** Questa impostazione indica al framework di eseguire i file SQL ogni volta che l'applicazione viene avviata. Sebbene sia utile in fase di sviluppo e per le demo, il video sottolinea che per la produzione esistono alternative più strutturate come **Flyway** o **Liquibase**.

## Workflow di esecuzione

Il processo segue un ordine preciso: una volta stabilita la connessione al database (PostgreSQL in questo caso quindi le impostazioni precedenti), Spring Boot esegue prima lo script `schema.sql` per creare la struttura e, subito dopo, lo script `data.sql` per inserire i record.

## Verifica dei risultati

Il video mostra come verificare l'avvenuta inizializzazione utilizzando uno strumento esterno come **DBeaver** (un client database gratuito).

Per collegarsi al database da DBeaver, appena installato, accedi a Nuova Connessione (Ctrl+Maiusc+N), scegli PostgreSQL, dove di base le impostazioni sono tutte giuste e immetti solo la password che è stata impostata sull'`application.properties`. In caso è possibile fare il test della connessione che è utile per determinare anche se i plugin necessari sono installati.

Collegandosi all'istanza PostgreSQL, è possibile visualizzare graficamente la nuova tabella (es. `widgets`) e confermare che i dati inseriti tramite lo script SQL siano effettivamente presenti.

Questa parte conclude la gestione "manuale" del database tramite SQL.

## Databases Part 2 - Spring JDBC (SQL)

Anche questa parte viene messa molto come strumento didattico e di contesto. Serve a mostrarti cosa succede sotto la scocca e quanto codice ripetitivo (boilerplate) e fatica siano necessari per gestire manualmente le query SQL, la mappatura dei risultati con i RowMapper e la gestione delle connessioni.

Quindi non ci soffermeremo troppo nel setup del testing ecc... anche perché dopo passeremo alla Spring Data JPA, un'API di alto livello (JPA/Hibernate) orientata agli oggetti. **Va SOTTOLINEATO come tutte queste sezioni non siano inutili, anzi spiegano al meglio come spring e successivamente JPA semplifichi la vita di molti sviluppatori tutti i giorni.**

### JdbcTemplate Setup

*Lo strumento fondamentale di Spring per interagire con il database tramite SQL in modo semplificato.*

Cos'è JdbcTemplate?

**JdbcTemplate** è un componente del framework Spring che agisce come un'evoluzione del classico JDBC. Il suo scopo principale è quello di eliminare il codice ripetitivo (*boilerplate*) necessario per gestire le connessioni al database, l'apertura e la chiusura degli statement e la gestione delle eccezioni, lasciando allo sviluppatore solo il compito di definire la query SQL e i parametri.

### Creazione della Classe di Configurazione

Per rendere disponibile **JdbcTemplate** nell'applicazione, il video mostra come procedere con una configurazione esplicita basata su classi Java:

- Si crea un pacchetto dedicato (es. **config**) e una classe (es. **DatabaseConfig**).
- La classe deve essere annotata con **@Configuration**, segnalando a Spring che contiene definizioni di componenti (Bean) da gestire.

## Definizione del Bean JdbcTemplate

All'interno della classe di configurazione, si definisce un metodo per creare l'istanza di `JdbcTemplate`:

- **Annotazione @Bean**: Il metodo deve essere annotato con `@Bean` per registrare l'oggetto restituito nel contesto dell'applicazione.
- **Dipendenza dal DataSource**: Il metodo richiede un oggetto `DataSource` come argomento. Spring, grazie alla Dependency Injection, individuerà automaticamente il `DataSource` configurato (sia esso H2 o PostgreSQL) e lo passerà al metodo.
- **Istanziamento**: Il metodo restituisce semplicemente una `new JdbcTemplate(dataSource)`.

```
@Configuration
public class DatabaseConfig {

    @Bean
    public JdbcTemplate jdbcTemplate(final DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }
}
```

## Utilizzo tramite Dependency Injection

Una volta configurato come Bean, `JdbcTemplate` è pronto per essere utilizzato in altri componenti, tipicamente nei **DAO** (Data Access Objects):

Questa configurazione permette di iniziare a scrivere query SQL dirette all'interno dei tuoi servizi di persistenza.

Per la connessione al database invece abbiamo questa configurazione nel `DatabaseApplication`, molto semplice:

```
@SpringBootApplication
@Log
public class DatabaseApplication {

    public static void main(String[] args) {
        SpringApplication.run(DatabaseApplication.class, args);
    }
}
```

## What is a DAO?

Adesso affrontiamo la teoria e la logica dietro il pattern **Data Access Object**, essenziale per mantenere il codice pulito e modulare quando si lavora con i database.

### Il Problema: Duplicazione e Accoppiamento

Senza un DAO, ogni servizio che deve interagire con il database dovrebbe conoscere la struttura esatta delle tabelle, scrivere le proprie query SQL e gestire manualmente la conversione tra i dati SQL e gli oggetti Java. Se più servizi avessero bisogno degli stessi dati (ad esempio, informazioni sugli autori), ti ritroveresti con una massiccia duplicazione di codice difficile da mantenere.

### La Soluzione: Il Pattern DAO

Un DAO è una classe dedicata che ha il compito esclusivo di gestire le interazioni con una specifica entità del database.

- **Responsabilità:** Conosce la struttura del database e gestisce tutte le mappature da SQL a oggetti Java e viceversa.
- **Interfaccia Pulita:** Espone ai livelli superiori (come il Service Layer) un'interfaccia semplice e pulita, nascondendo la complessità tecnica del database.

### Esempio Pratico: Libri e Autori

Il video utilizza l'esempio di un sistema editoriale:

- **Entità:** Abbiamo **Book** (Libro) e **Author** (Autore).
- **Relazione:** Un autore può scrivere molti libri, ma un libro ha un solo autore.
- **Implementazione:** Creeremo un **BookDao** e un **AuthorDao**. In questo modo, se un servizio ha bisogno di un libro, non scriverà la query SQL, ma chiamerà semplicemente il metodo del **BookDao**, che restituirà un oggetto Java già pronto all'uso.

### Vantaggi Architetture

L'uso dei DAO garantisce una netta **separazione tra il Service Layer e il Persistence Layer**. Questa modularità rende l'applicazione molto più facile da scrivere, mantenere e, soprattutto, testare.

## Set Up a DAO

In questa sezione vengono descritti i passi pratici necessari per preparare l'architettura del progetto prima di scrivere la logica di accesso ai dati.

### Definizione dello Schema SQL

Il primo passo consiste nel definire la struttura delle tabelle nel database. Viene creato un file **schema.sql** in **src/main/resources** per generare le tabelle necessarie:

- **Tabella **authors****: Include i campi **ID** (chiave primaria), **name** e **age**.
- **Tabella **books****: Include **ISBN** (chiave primaria), **title** e **author\_id**.

```
DROP TABLE IF EXISTS "books";
DROP TABLE IF EXISTS "authors";

CREATE TABLE "authors" (
  "id" bigint DEFAULT nextval('authors_id_seq') NOT NULL,
  "name" text,
  "age" integer,
  CONSTRAINT "authors_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "books" (
  "isbn" text NOT NULL,
  "title" text,
  "author_id" bigint,
  CONSTRAINT "books_pkey" PRIMARY KEY ("isbn"),
  CONSTRAINT "fk_author" FOREIGN KEY(author_id)
  REFERENCES authors(id)
);
```

- **Vincoli**: Viene impostata una *Foreign Key* che collega **author\_id** della tabella libri all'ID della tabella autori, garantendo l'integrità dei dati.
- **Configurazione**: In **application.properties**, si imposta **spring.sql.init.mode=always** per assicurarsi che lo schema venga eseguito a ogni avvio (utile in fase di sviluppo).

```
spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=changemeinprod!
spring.datasource.driver-class-name=org.postgresql.Driver

spring.sql.init.mode=always
```

## Creazione degli Oggetti di Dominio (Entities)

Vengono create le classi Java che rappresentano i dati nel database, utilizzando **Lombok** per ridurre il codice ripetitivo:

- **Classe Author**: Contiene **ID** (Long), **name** (String) e **age** (Integer). Si utilizzano le annotazioni **@Data**, **@AllArgsConstructor**, **@NoArgsConstructor** e **@Builder**.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class Author {
    private Long id;
    private String name;
    private Integer age;
}
```

- **Classe Book**: Contiene **ISBN** (String), **title** (String) e **authorId** (Long). In questa fase, la classe mappa direttamente la struttura della tabella: si usa un **authorId** numerico invece di un oggetto **Author** annidato per riflettere esattamente come i dati sono memorizzati nel database relazionale.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class Book {
    private String isbn;
    private String title;
    private Long authorId;
}
```

## Struttura del Package DAO

L'organizzazione del codice segue una convenzione precisa per favorire la modularità:

- Si crea un package principale chiamato **dao** che contiene le interfacce (**AuthorDao** e **BookDao**). Queste interfacce definiscono i metodi che saranno disponibili per il resto dell'applicazione, di conseguenza per il momento rimarranno vuote.
- All'interno del package **dao**, si crea una sottocartella **impl** per le implementazioni concrete (es. **AuthorDaoImpl**), mantenendo separata la definizione del contratto dalla sua realizzazione tecnica.

## Implementazione e Iniezione di JdbcTemplate

Nelle classi di implementazione (come `AuthorDaoImpl`), viene configurato l'accesso al database:

- **Iniezione del Costruttore:** Si dichiara una variabile d'istanza per `JdbcTemplate`. Spring inietterà automaticamente il Bean configurato nel costruttore della classe.
- **Immutabilità:** Poiché si utilizza l'iniezione tramite costruttore, è possibile dichiarare il campo `JdbcTemplate` come **final**. Questo assicura che la dipendenza non venga riassegnata dopo l'istanziamento, rendendo il codice più sicuro e facile da testare.

```
public class AuthorDaoImpl implements AuthorDao {  
  
    private final JdbcTemplate jdbcTemplate;  
  
    public AuthorDaoImpl(final JdbcTemplate jdbcTemplate) {  
        this.jdbcTemplate = jdbcTemplate;  
    }  
}
```

```
public class BookDaoImpl implements BookDao {  
  
    private final JdbcTemplate jdbcTemplate;  
  
    public BookDaoImpl(final JdbcTemplate jdbcTemplate) {  
        this.jdbcTemplate = jdbcTemplate;  
    }  
}
```

Questa fase di setup è fondamentale perché crea lo "scheletro" su cui verranno poi costruite le operazioni CRUD (Create, Read, Update, Delete).

## Create DAOs

In questa fase della documentazione, ci occupiamo dell'implementazione pratica della funzione **Create** (la "C" dell'acronimo CRUD) per inserire dati nel database utilizzando il pattern DAO.

### Sviluppo tramite Unit Test

Il video adotta un approccio orientato ai test (TDD) per garantire che i DAO generino l'SQL corretto senza dover ancora avviare l'intero database.

- **Setup con Mockito:** Viene creato un test unitario (es. `AuthorDaoImplTests`) utilizzando l'annotazione `@Mock` per simulare il `JdbcTemplate` e `@InjectMocks` per iniettarlo nella classe da testare.
- **Verifica dell'SQL:** Si utilizza il metodo `Mockito.verify()` per controllare che il metodo `update` del `JdbcTemplate` venga effettivamente chiamato con la query SQL e i parametri previsti.
- Dato che stiamo procedendo con la programmazione Test Oriented, specifico anche che la classe `DatabaseApplicationTests` è sviluppata come:

```
package com.devtiro.database;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
@SpringBootTest
class DatabaseApplicationTests {
    @Test
    void contextLoads() {
    }
}
```

Per l'`application.properties` della sezione test invece avremo:

```
spring.datasource.url=jdbc:h2:mem:testdb;MODE=PostgreSQL;DATABASE_TO
_LOWER=TRUE;DEFAULT_NULL_ORDERING=HIGH
spring.datasource.username=sa
spring.datasource.password=password
spring.datasource.driver-class-name=org.h2.Driver
```

## Implementazione per l'Autore (AuthorDao)

Per inserire un autore nel database, il processo segue questi passaggi tecnici:

- **Definizione del metodo:** Il metodo `create(Author author)` viene aggiunto all'interfaccia e poi implementato nella classe `AuthorDaoImpl`.

```
@Override
public void create(Author author) {
    jdbcTemplate.update(
        "INSERT INTO authors (id, name, age) VALUES (?, ?, ?)",
        author.getId(), author.getName(), author.getAge()
    );
}
```

- **La Query SQL:** Si utilizza un'istruzione standard: `INSERT INTO authors (id, name, age) VALUES (?, ?, ?)`.
- **Esecuzione:** Dentro `AuthorDaoImplTests` si richiama `jdbcTemplate.update(sql, author.getId(), author.getName(), author.getAge())`, passando i valori estratti dall'oggetto Java.

```
@ExtendWith(MockitoExtension.class)
public class AuthorDaoImplTests {

    @Mock
    private JdbcTemplate jdbcTemplate;

    @InjectMocks
    private AuthorDaoImpl underTest;

    @Test
    public void testThatCreateAuthorGeneratesCorrectSql() {
        Author author = Author.builder()
            .id(1L)
            .name("Abigail Rose")
            .age(80)
            .build();

        underTest.create(author);

        verify(jdbcTemplate).update(
            eq("INSERT INTO authors (id, name, age) VALUES (?, ?, ?)"),
            eq(1L), eq("Abigail Rose"), eq(80)
        );
    }
}
```

## Implementazione per il Libro (BookDao)

Il processo per i libri è speculare a quello degli autori, adattando i campi alla struttura della tabella dedicata:

- **Query specifica:** L'SQL aggiunto in `BookDaoImpl` e all'interfaccia è `INSERT INTO books (isbn, title, author_id) VALUES (?, ?, ?)`.

```
@Override
public void create(Book book) {
    jdbcTemplate.update(
        "INSERT INTO books (isbn, title, author_id) VALUES (?, ?, ?)",
        book.getIsbn(),
        book.getTitle(),
        book.getAuthorId()
    );
}
```

- **Mappatura dei campi:** In `BookDaoImplTests` vengono passati l'ISBN, il titolo e l'ID dell'autore (come riferimento esterno) al metodo `update`.

```
@ExtendWith(MockitoExtension.class)
public class BookDaoImplTests {

    @Mock
    private JdbcTemplate jdbcTemplate;

    @InjectMocks
    private BookDaoImpl underTest;

    @Test
    public void testThatCreateBookGeneratesCorrectSql() {
        Book book = Book.builder()
            .isbn("978-1-2345-6789-0")
            .title("The Shadow in the Attic")
            .authorId(1L)
            .build();

        underTest.create(book);

        verify(jdbcTemplate).update(
            eq("INSERT INTO books (isbn, title, author_id) VALUES (?, ?, ?)"),
            eq("978-1-2345-6789-0"),
            eq("The Shadow in the Attic"),
            eq(1L)
        );
    }
}
```

## Sicurezza e SQL Injection

Un punto cruciale sottolineato nel video è la protezione dei dati tramite i **Prepared Statements**.

- **I punti di domanda (?)**: Invece di concatenare direttamente i valori nella stringa SQL (pratica pericolosa), si usano i segnaposto.
- **Vantaggio**: Questo approccio protegge l'applicazione da attacchi di **SQL Injection**, poiché il framework gestisce in modo sicuro l'inserimento dei parametri nel comando SQL.

## Implement Read One

Passiamo al come recuperare un singolo record dal database utilizzando `JdbcTemplate` e trasformarlo in un oggetto Java.

Il concetto di "Read One"

Questa operazione (spesso chiamata `findOne` o `findById`) consiste nel passare un identificativo unico (come l'ID dell'autore o l'ISBN del libro) per ottenere l'entità corrispondente. Il video suggerisce di restituire un `Optional<T>`: se il record esiste viene restituito l'oggetto, altrimenti si riceve un `Optional.empty`, evitando così i pericolosi `null`.

L'utilizzo di `RowMapper`

Poiché JDBC restituisce i dati in un formato tabellare (`ResultSet`), è necessario un componente che "mappi" queste righe in oggetti Java.

- **Implementazione:** Si crea una classe interna (es. `AuthorRowMapper`) che implementa l'interfaccia `RowMapper`.

```
public static class AuthorRowMapper implements RowMapper<Author> {  
  
    @Override  
    public Author mapRow(ResultSet rs, int rowNum) throws  
SQLException {  
        return Author.builder()  
            .id(rs.getLong("id"))  
            .name(rs.getString("name"))  
            .age(rs.getInt("age"))  
            .build();  
    }  
}
```

```
public static class BookRowMapper implements RowMapper<Book> {  
  
    @Override  
    public Book mapRow(ResultSet rs, int rowNum) throws SQLException  
{  
        return Book.builder()  
            .isbn(rs.getString("isbn"))  
            .title(rs.getString("title"))  
            .authorId(rs.getLong("author_id"))  
            .build();  
    }  
}
```

- **Logica:** Il metodo `mapRow` estrae i valori dalle colonne SQL (es. `rs.getString("name")`) e li inserisce nei campi dell'oggetto tramite un Builder o i setter.

## Esecuzione della Query con JdbcTemplate

A differenza della creazione dove si usa `.update()`, per la lettura si utilizza il metodo `.query()`.

- **Sintassi:** Si passa la stringa SQL (es. `SELECT id, name, age FROM authors WHERE id = ?`), una nuova istanza del `RowMapper` e l'ID cercato.
- **Risultato:** Il metodo restituisce una lista. Per ottenere un singolo `Optional`, si usa l'API Stream: `results.stream().findFirst()`.

Dove per Author:

- In `AuthorDaoImpl`:

```
@Override
public Optional<Author> findOne(long authorId) {
    List<Author> results = jdbcTemplate.query(
        "SELECT id, name, age FROM authors WHERE id = ? LIMIT 1",
        new AuthorRowMapper(), authorId);

    return results.stream().findFirst();
}
```

- In `AuthorDaoImplTests`:

```
@Test
public void testThatFindOneGeneratesTheCorrectSql() {
    underTest.findOne(1L);
    verify(jdbcTemplate).query(
        eq("SELECT id, name, age FROM authors WHERE id = ? LIMIT 1"),
        ArgumentMatchers.<AuthorDaoImpl.AuthorRowMapper>any(),
        eq(1L)
    );
}
```

E per Book:

- In BookDaoImpl:

```
@Override
public Optional<Book> findOne(String isbn) {
    List<Book> results = jdbcTemplate.query(
        "SELECT isbn, title, author_id from books WHERE isbn = ?
LIMIT 1",
        new BookRowMapper(),
        isbn
    );
    return results.stream().findFirst();
}
```

- In BookDaoImplTests:

```
@Test
public void testThatFindOneBookGeneratesCorrectSql() {
    underTest.findOne("978-1-2345-6789-0");
    verify(jdbcTemplate).query(
        eq("SELECT isbn, title, author_id from books WHERE isbn =
? LIMIT 1"),
        ArgumentMatchers.<BookDaoImpl.BookRowMapper>any(),
        eq("978-1-2345-6789-0")
    );
}
```

## Dettaglio SQL e Sicurezza

- **Parametri:** Si usano sempre i segnaposto `?` per prevenire attacchi di SQL Injection.
- **Ottimizzazione:** Nelle query di lettura singola, viene spesso aggiunto `LIMIT 1` alla fine del comando SQL per garantire che il database si fermi al primo risultato trovato.

## Find Many

Qui spieghiamo come recuperare l'intera lista di record (es. tutti gli autori o tutti i libri) dal database e come gestire correttamente l'ambiente di test per evitare interferenze tra i dati.

### Il Concetto di "Find Many"

Questa operazione (spesso definita come `find()` o `findAll()`) ha lo scopo di restituire una lista di tutti gli oggetti di un certo tipo presenti nel database. A differenza di "Read One", non riceve parametri di ricerca (come un ID) e restituisce sempre una lista (che sarà vuota se non ci sono record), evitando il ritorno di valori nulli.

### Test Unitario e Implementazione

Il processo segue lo schema già visto per le altre operazioni:

- **SQL Query:** Per gli autori, la query è `SELECT id, name, age FROM authors`. Per i libri, è `SELECT isbn, title, author_id FROM books`.
- **Utilizzo di `RowMapper`:** Si utilizza `jdbcTemplate.query()`, passando la query SQL e un'istanza del `RowMapper` specifico (es. `AuthorRowMapper`) che si occupa di trasformare ogni riga del database in un oggetto Java.
- **Risultato:** Il metodo restituisce un oggetto `List<T>` contenente tutte le entità mappate.

### Test di Integrazione

Per verificare che il codice funzioni con un database reale (H2 in memoria):

- **Setup:** Si creano più oggetti di test (es. Autore A, B e C) e si salvano nel database.
- **Asserzione:** Si chiama il metodo `find()` e si verifica, tramite librerie come **AssertJ**, che la lista risultante abbia la dimensione corretta (es. `hasSize(3)`) e contenga esattamente gli oggetti creati.

### Gestione della "Test Pollution"

Un problema comune nei test di integrazione è la "contaminazione dei test" (*Test Pollution*): i dati inseriti da un test rimangono nel database e possono far fallire i test successivi (ad esempio causando errori di chiave duplicata).

- **La Soluzione:** L'annotazione `@DirtiesContext(classMode = AfterEachTestMethod)`.
- **Funzionamento:** Questa istruzione forza Spring a ripulire completamente il contesto dell'applicazione e a resettare il database dopo l'esecuzione di ogni singolo metodo di test, garantendo che ogni test parta da una base pulita.

- In AuthorDaoImpl:

```
@Override
public List<Author> find() {
    return jdbcTemplate.query(
        "SELECT id, name, age FROM authors",
        new AuthorRowMapper()
    );
}
```

- In AuthorDaoImplTests:

```
@Test
public void testThatFindManyGeneratesCorrectSql() {
    underTest.find();
    verify(jdbcTemplate).query(
        eq("SELECT id, name, age FROM authors"),
        ArgumentMatchers.<AuthorDaoImpl.AuthorRowMapper>any()
    );
}
```

E per Book:

- In BookDaoImpl:

```
@Override
public List<Book> find() {
    return jdbcTemplate.query(
        "SELECT isbn, title, author_id from books",
        new BookRowMapper()
    );
}
```

- In BookDaoImplTests:

```
@Test
public void testThatFindGeneratesCorrectSql() {
    underTest.find();
    verify(jdbcTemplate).query(
        eq("SELECT isbn, title, author_id from books"),
        ArgumentMatchers.<BookDaoImpl.BookRowMapper>any()
    );
}
```

## Full Update

Adesso passiamo al grande e come implementare l'aggiornamento completo di un record esistente utilizzando il pattern DAO e `JdbcTemplate`.

### Definizione di "Full Update"

Un aggiornamento completo (Full Update) sostituisce tutti i valori di una riga nel database con i nuovi dati forniti dall'oggetto Java. A differenza di un aggiornamento parziale, qui si inviano tutti i campi (ID, nome, età, ecc.).

### Implementazione per l'Autore

- **Firma del metodo:** Il metodo `update` accetta l'ID dell'autore da modificare e l'oggetto `Author` con i nuovi dati: `void update(long id, Author author)`.
- **SQL Query:** Utilizza il comando `UPDATE`: `UPDATE authors SET id = ?, name = ?, age = ? WHERE id = ?`.
- **Esecuzione:** Si usa `jdbcTemplate.update()`, passando i valori dell'oggetto e l'ID per la clausola `WHERE` per identificare correttamente il record.

### Implementazione per il Libro

Il processo è analogo, ma utilizza l'ISBN come identificatore unico:

- **SQL Query:** `UPDATE books SET isbn = ?, title = ?, author_id = ? WHERE isbn = ?`.
- **Sicurezza:** Come per la creazione, si usano i segnaposto (?) per i parametri per prevenire attacchi di SQL Injection.

### Verifica tramite Test

- **Unit Test:** Si usa Mockito per verificare che il DAO generi la stringa SQL corretta e passi i parametri nell'ordine giusto al `JdbcTemplate`.
- **Integration Test:** Si crea un record, se ne modifica il nome o il titolo in Java, si chiama `update()` e infine si usa `findOne()` per confermare che le modifiche siano state salvate nel database reale.

- In AuthorDaoImpl:

```
@Override
public void update(long id, Author author) {
    jdbcTemplate.update(
        "UPDATE authors SET id = ?, name = ?, age = ? WHERE id = ?",
        author.getId(), author.getName(), author.getAge(), id
    );
}
```

- In AuthorDaoImplTests:

```
@Test
public void testThatUpdateGeneratesCorrectSql() {
    Author author = TestDataUtil.createTestAuthorA();
    underTest.update(3L, author);

    verify(jdbcTemplate).update(
        "UPDATE authors SET id = ?, name = ?, age = ? WHERE id = ?",
        1L, "Abigail Rose", 80, 3L
    );
}
```

E per Book:

- In BookDaoImpl:

```
@Override
public void update(String isbn, Book book) {
    jdbcTemplate.update(
        "UPDATE books SET isbn = ?, title = ?, author_id = ? WHERE isbn = ?",
        book.getIsbn(), book.getTitle(), book.getAuthorId(), isbn
    );
}
```

- In BookDaoImplTests:

```
@Test
public void testThatUpdateGeneratesCorrectSql() {
    Book book = TestDataUtil.createTestBookA();
    underTest.update("978-1-2345-6789-0", book);
    verify(jdbcTemplate).update(
        "UPDATE books SET isbn = ?, title = ?, author_id = ? WHERE isbn = ?",
        "978-1-2345-6789-0", "The Shadow in the Attic", 1L,
        "978-1-2345-6789-0"
    );
}
```

## Implement Delete

L'ultima fase del ciclo CRUD nel pattern DAO riguarda la rimozione dei record dal database, completando così la logica di persistenza manuale tramite SQL.

### Implementazione per l'Autore (AuthorDao)

Per eliminare un autore, il metodo `delete(long id)` viene definito nell'interfaccia e implementato nella classe concreta utilizzando `JdbcTemplate`.

- **Query SQL:** Si utilizza il comando `DELETE FROM authors WHERE id = ?`.
- **Esecuzione:** Il metodo `jdbcTemplate.update()` esegue la query passando l'ID come parametro, garantendo la sicurezza contro attacchi di SQL Injection grazie all'uso dei prepared statements.

### Implementazione per il Libro (BookDao)

Il processo per i libri segue la stessa logica, ma utilizza l'ISBN come identificatore.

- **Query SQL:** `DELETE FROM books WHERE isbn = ?`.
- **Gestione Vincoli:** Durante i test di integrazione, viene ricordato che i libri hanno un vincolo di integrità (foreign key) verso gli autori; pertanto, un libro deve essere associato a un autore esistente prima di poter essere manipolato.

## Verifica e Testing

Il video descrive un doppio approccio per garantire la validità dell'operazione:

- **Unit Test (con Mockito):** Si verifica che il `JdbcTemplate` riceva la chiamata `update` con la stringa SQL corretta e l'argomento ID previsto.
- **Test di Integrazione:** Viene eseguito un flusso completo dove un record viene creato, eliminato e infine cercato tramite `findOne()`. Il test è superato se il risultato restituito è un `Optional.empty`.

- In `AuthorDaoImpl`:

```
@Override
public void delete(long id) {
    jdbcTemplate.update(
        "DELETE FROM authors where id = ?",
        id
    );
}
```

- In AuthorDaoImplTests:

```
@Test
public void testThatDeleteGeneratesTheCorrectSql() {
    underTest.delete(1L);
    verify(jdbcTemplate).update(
        "DELETE FROM authors where id = ?",
        1L
    );
}
```

E per Book:

- In BookDaoImpl:

```
@Override
public void delete(String isbn) {
    jdbcTemplate.update(
        "DELETE FROM books where isbn = ?",
        isbn
    );
}
```

- In BookDaoImplTests:

```
@Test
public void testThatDeleteGeneratesCorrectSql() {
    underTest.delete("978-1-2345-6789-0");
    verify(jdbcTemplate).update(
        "DELETE FROM books where isbn = ?",
        "978-1-2345-6789-0"
    );
}
```

Verso l'automazione: JPA e Hibernate

Con l'implementazione del `delete`, si conclude la gestione manuale del layer di persistenza. L'autore del video osserva come questo approccio, pur essendo granulare, richieda una quantità notevole di codice ripetitivo (*boilerplate*) e query SQL scritte a mano.

Questa sezione chiude il capitolo sui DAO.

## Create Integration Test

Questa sezione descrive il passaggio fondamentale dal testing isolato (unit test con mock) alla verifica del codice in un ambiente che simula il database reale. Molto utile.

### Obiettivo del Test di Integrazione

Mentre gli unit test verificano solo che venga generata la stringa SQL corretta, i test di integrazione eseguono effettivamente tale SQL contro un database per confermare che la comunicazione, i tipi di dato e i vincoli siano gestiti correttamente.

### Configurazione con H2 (In-Memory Database)

Per mantenere i test veloci e indipendenti, si utilizza il database in memoria **H2**.

- **PostgreSQL Mode:** Viene configurato in `src/test/resources/application.properties` affinché emuli le caratteristiche specifiche di PostgreSQL. Questo garantisce che l'SQL scritto per il database di produzione sia compatibile con l'ambiente di test.
- **Annotazioni Chiave:** Si usa `@SpringBootTest` per avviare il contesto dell'applicazione e `@ExtendWith(SpringExtension.class)` per abilitare il supporto di Spring nei test JUnit.

### Iniezione delle Dipendenze nei Test

Nei test di integrazione, a differenza del codice di produzione, l'iniezione tramite costruttore richiede l'annotazione esplicita `@Autowired` sopra il costruttore della classe di test affinché Spring sappia quali Bean (come `AuthorDaoImpl`) deve iniettare.

### Gestione della "Test Pollution" con `@DirtiesContext`

Un problema critico nei test di database è che i dati inseriti da un test possono influenzare i risultati di quelli successivi.

- **Soluzione:** Si applica l'annotazione `@DirtiesContext(classMode = AfterEachTestMethod)`. Questa forza Spring a ripulire il contesto dell'applicazione e resettare il database dopo ogni singolo metodo di test, garantendo che ogni test parta da una base "pulita".

### Flusso di Esecuzione: Create & Recall

Il test viene strutturato seguendo questo schema logico:

- **Setup:** Si genera un oggetto di test (es. un Autore) utilizzando una classe di utilità chiamata `TestDataUtil` per favorire il riutilizzo del codice.
- **Esecuzione:** Si chiama il metodo `create()` del DAO per persistere l'oggetto.

- **Verifica:** Si usa il metodo `findOne()` per recuperare l'oggetto dal database tramite il suo ID. Infine, si utilizza **AssertJ** per verificare che l'oggetto recuperato sia identico a quello inviato.

Ottenendo per `AuthorDaoImplIntegrationTests`:

```
package com.devtiro.database.dao.impl;

import com.devtiro.database.TestDataUtil;
import com.devtiro.database.domain.Author;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.annotation.DirtiesContext;
import org.springframework.test.context.junit.jupiter.SpringExtension;

import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest
@ExtendWith(SpringExtension.class)
@DirtiesContext(classMode =
DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD)
public class AuthorDaoImplIntegrationTests {

    private AuthorDaoImpl underTest;

    @Autowired
    public AuthorDaoImplIntegrationTests(AuthorDaoImpl underTest) {
        this.underTest = underTest;
    }

    @Test
    public void testThatAuthorCanBeCreatedAndRecalled() {
        Author author = TestDataUtil.createTestAuthorA();
        underTest.create(author);
        Optional<Author> result = underTest.findOne(author.getId());
        assertThat(result).isPresent();
        assertThat(result.get()).isEqualTo(author);
    }
}
```

```

@Test
public void testThatMultipleAuthorsCanBeCreatedAndRecalled() {
    Author authorA = TestDataUtil.createTestAuthorA();
    underTest.create(authorA);
    Author authorB = TestDataUtil.createTestAuthorB();
    underTest.create(authorB);
    Author authorC = TestDataUtil.createTestAuthorC();
    underTest.create(authorC);

    List<Author> result = underTest.find();
    assertThat(result)
        .hasSize(3)
        .containsExactly(authorA, authorB, authorC);
}

@Test
public void testThatAuthorCanBeUpdated() {
    Author authorA = TestDataUtil.createTestAuthorA();
    underTest.create(authorA);
    authorA.setName("UPDATED");
    underTest.update(authorA.getId(), authorA);
    Optional<Author> result = underTest.findOne(authorA.getId());
    assertThat(result).isPresent();
    assertThat(result.get()).isEqualTo(authorA);
}

@Test
public void testThatAuthorCanBeDeleted() {
    Author authorA = TestDataUtil.createTestAuthorA();
    underTest.create(authorA);
    underTest.delete(authorA.getId());
    Optional<Author> result = underTest.findOne(authorA.getId());
    assertThat(result).isEmpty();
}
}

```

Per `BookDaoImplIntegrationTests`:

```
package com.devtiro.database.dao.impl;

import com.devtiro.database.TestDataUtil;
import com.devtiro.database.dao.AuthorDao;
import com.devtiro.database.domain.Author;
import com.devtiro.database.domain.Book;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.annotation.DirtiesContext;
import org.springframework.test.context.junit.jupiter.SpringExtension;

import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest
@ExtendWith(SpringExtension.class)
@DirtiesContext(classMode =
DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD)
public class BookDaoImplIntegrationTests {

    private AuthorDao authorDao;
    private BookDaoImpl underTest;

    @Autowired
    public BookDaoImplIntegrationTests(BookDaoImpl underTest, AuthorDao
authorDao) {
        this.underTest = underTest;
        this.authorDao = authorDao;
    }

    @Test
    public void testThatBookCanBeCreatedAndRecalled() {
        Author author = TestDataUtil.createTestAuthorA();
        authorDao.create(author);
        Book book = TestDataUtil.createTestBookA();
        book.setAuthorId(author.getId());
        underTest.create(book);
        Optional<Book> result = underTest.findOne(book.getIsbn());
        assertThat(result).isPresent();
        assertThat(result.get()).isEqualTo(book);
    }
}
```

```
@Test
public void testThatMultipleBooksCanBeCreatedAndRecalled() {
    Author author = TestDataUtil.createTestAuthorA();
    authorDao.create(author);

    Book bookA = TestDataUtil.createTestBookA();
    bookA.setAuthorId(author.getId());
    underTest.create(bookA);

    Book bookB = TestDataUtil.createTestBookB();
    bookB.setAuthorId(author.getId());
    underTest.create(bookB);

    Book bookC = TestDataUtil.createTestBookC();
    bookC.setAuthorId(author.getId());
    underTest.create(bookC);

    List<Book> result = underTest.find();
    assertThat(result)
        .hasSize(3)
        .containsExactly(bookA, bookB, bookC);
}

@Test
public void testThatBookCanBeUpdated() {
    Author author = TestDataUtil.createTestAuthorA();
    authorDao.create(author);

    Book bookA = TestDataUtil.createTestBookA();
    bookA.setAuthorId(author.getId());
    underTest.create(bookA);

    bookA.setTitle("UPDATED");
    underTest.update(bookA.getIsbn(), bookA);

    Optional<Book> result = underTest.findOne(bookA.getIsbn());
    assertThat(result).isPresent();
    assertThat(result.get()).isEqualTo(bookA);
}
```

```

@Test
public void testThatBookCanBeDeleted() {
    Author author = TestDataUtil.createTestAuthorA();
    authorDao.create(author);

    Book bookA = TestDataUtil.createTestBookA();
    bookA.setAuthorId(author.getId());
    underTest.create(bookA);

    underTest.delete(bookA.getIsbn());

    Optional<Book> result = underTest.findOne(bookA.getIsbn());
    assertThat(result).isEmpty();
}
}

```

Infine per il citato `TestDataUtil`:

```

package com.devtiro.database;

import com.devtiro.database.domain.Author;
import com.devtiro.database.domain.Book;

public final class TestDataUtil {
    private TestDataUtil(){
    }

    public static Author createTestAuthorA() {
        return Author.builder()
            .id(1L)
            .name("Abigail Rose")
            .age(80)
            .build();
    }

    public static Author createTestAuthorB() {
        return Author.builder()
            .id(2L)
            .name("Thomas Cronin")
            .age(44)
            .build();
    }

    public static Author createTestAuthorC() {
        return Author.builder()
            .id(3L)
            .name("Jesse A Casey")
            .age(24)
            .build();
    }
}

```

```
public static Book createTestBookA() {  
    return Book.builder()  
        .isbn("978-1-2345-6789-0")  
        .title("The Shadow in the Attic")  
        .authorId(1L)  
        .build();  
}  
  
public static Book createTestBookB() {  
    return Book.builder()  
        .isbn("978-1-2345-6789-1")  
        .title("Beyond the Horizon")  
        .authorId(1L)  
        .build();  
}  
  
public static Book createTestBookC() {  
    return Book.builder()  
        .isbn("978-1-2345-6789-2")  
        .title("The Last Ember")  
        .authorId(1L)  
        .build();  
}  
}
```

Questa sezione assicura che il layer di persistenza sia solido prima di procedere.

## Databases Part 3 - Spring Data JPA (Objects)

### Spring Data JPA Setup

Da questa parte in poi vi è una transizione fondamentale: l'abbandono della gestione manuale del database tramite SQL e DAO a favore di un'API di alto livello che automatizza il lavoro.

#### Rimozione del codice "Boilerplate"

Spring Data JPA e Hibernate riducono drasticamente la quantità di codice necessaria. Per questo motivo, il setup inizia con la pulizia delle componenti create nei capitoli precedenti:

- **Eliminazione di `schema.sql`:** Hibernate è in grado di generare automaticamente la struttura delle tabelle partendo dalle classi Java, rendendo superfluo lo script SQL manuale per la creazione dello schema.
- **Rimozione del layer DAO:** Poiché Spring Data JPA genererà automaticamente la logica di accesso ai dati, l'intero pacchetto `dao` (interfacce e implementazioni SQL) viene rimosso.
- **Eliminazione della configurazione `JdbcTemplate`:** La classe di configurazione che definiva il Bean `JdbcTemplate` viene eliminata, poiché non interagiranno più direttamente con SQL a basso livello.

#### Aggiornamento delle dipendenze nel pom.xml

Il motore di questa automazione risiede in un nuovo "starter". Nel file di configurazione Maven:

- La dipendenza `spring-boot-starter-jdbc` viene sostituita con `spring-boot-starter-data-jpa`.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

- Questo pacchetto trascina con sé tutte le librerie necessarie, tra cui Hibernate (l'implementazione ORM predefinita) e le API per la gestione dei repository.

## Introduzione dei Repository

Invece di scrivere classi che implementano i metodi di salvataggio o ricerca, Spring Data JPA introduce il concetto di **Repository**:

- Viene creato un nuovo pacchetto `repositories`.
- Inizialmente si definiscono delle interfacce semplici, come `AuthorRepository` e `BookRepository`, che restano temporaneamente vuote. Sarà il framework, successivamente, a "riempirle" di funzionalità estendendo interfacce standard come `CrudRepository`.

Questo setup iniziale "pulisce" l'architettura e prepara il progetto per l'uso delle **Entities**.

## Create Entities

Descriviamo come trasformare i semplici oggetti Java (POJO) in **Entità JPA**, permettendo a Hibernate di mappare le classi direttamente sulle tabelle del database tramite annotazioni, con l'implementazione delle entità **Author** e **Book**:

### Trasformazione della classe Author

Per rendere la classe **Author** un'entità gestibile da Spring Data JPA, vengono applicate le seguenti configurazioni:

- **@Entity**: Etichetta la classe come entità JPA.
- **@Table(name = "authors")**: Specifica esplicitamente il nome della tabella nel database.
- **Gestione dell'ID**: Si utilizza **@Id** per marcare la chiave primaria. Per automatizzare la creazione degli ID, si usa **@GeneratedValue** con strategia **SEQUENCE**, definendo un generatore (es. **author\_id\_sequence**) che incrementa il valore a ogni nuovo inserimento.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
@Entity
@Table(name = "authors")
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"author_id_seq")
    private Long id;

    private String name;

    private Integer age;
}
```

## Trasformazione della classe Book

La classe `Book` segue una logica simile ma con alcune differenze strutturali:

- **Mappatura:** Viene annotata con `@Entity` e `@Table(name = "books")`.
- **ID Manuale:** A differenza dell'autore, per il libro si usa l'**ISBN** come `@Id`. In questo caso non si usa `@GeneratedValue` perché il codice ISBN viene fornito manualmente dallo sviluppatore e non generato dal database.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
@Entity
@Table(name="books")
public class Book {

    @Id
    private String isbn;

    private String title;

    @ManyToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "author_id")
    private Author author;
}
```

## Mappatura delle Relazioni (Many-to-One)

Uno dei passaggi più importanti è il passaggio da un semplice riferimento numerico (`authorId`) a un oggetto reale.

- **Riferimento all'Oggetto:** All'interno della classe `Book`, il campo `Long authorId` viene sostituito con un oggetto `Author author`.
- **@ManyToOne:** Questa annotazione definisce la relazione: molti libri possono appartenere a un solo autore.
- **@JoinColumn(name = "author\_id"):** Specifica il nome della colonna nel database che fungerà da chiave esterna (*foreign key*).
- **Cascading:** Impostando `CascadeType.ALL`, si istruisce Spring a propagare le operazioni: se salvi o modifichi un libro che contiene un nuovo autore, Hibernate salverà automaticamente anche l'autore nel database.

## Vantaggi dell'approccio a Entità

Grazie a queste annotazioni, Hibernate può analizzare le classi all'avvio e, se configurato con `ddl-auto: update`, creare o aggiornare automaticamente lo schema delle tabelle (colonne, tipi di dato e vincoli) senza bisogno di script SQL manuali.

Questa struttura centralizza la logica del database direttamente nel codice Java.

## Hibernate Auto DDL

Adesso vediamo come delegare a Hibernate la creazione automatica delle tabelle, eliminando la necessità di gestire manualmente file SQL come `schema.sql`.

### Il ruolo di Hibernate Auto DDL

Hibernate è in grado di analizzare le classi Java annotate con `@Entity` e generare i comandi DDL (*Data Definition Language*) necessari per allineare la struttura del database al codice. Questo garantisce che ogni modifica ai campi di una classe venga riflessa automaticamente nelle colonne della tabella corrispondente.

### Configurazione in `application.properties`

Per attivare questa funzione, si utilizza una proprietà specifica all'interno del file di configurazione:

- **Chiave:** `spring.jpa.hibernate.ddl-auto`.
- **Valore consigliato:** `update`. Con questo valore, Hibernate aggiorna lo schema esistente aggiungendo nuove tabelle o colonne senza distruggere i dati già presenti.

### Valori della proprietà

Oltre a `update`, esistono altre opzioni a seconda delle necessità:

- **`create`:** Crea lo schema all'avvio, cancellando ogni struttura precedente.
- **`create-drop`:** Crea lo schema all'avvio e lo elimina completamente allo spegnimento dell'applicazione (utile per i test).
- **`none`:** Non viene eseguita alcuna azione automatica.
- **`validate`:** Verifica solo che il database corrisponda alle entità, senza apportare modifiche.

`application.properties` ottenuto:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=changemeinprod!
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
```

## Risultato Pratico

Nel video viene mostrato come, dopo aver avviato l'app con la proprietà impostata su `update`, il database PostgreSQL (visualizzato tramite DBeaver) presenti automaticamente:

- La tabella **authors** con le colonne `id`, `age` e `name`.
- La tabella **books** con `isbn`, `title` e la chiave esterna `author_id`.
- La sequenza **author\_id\_sequence** per la gestione automatica degli ID degli autori.

## Implement Create & Read

Questa sezione mostra come l'ereditarietà dalle interfacce di Spring Data JPA elimini la necessità di scrivere manualmente la logica di persistenza, con l'implementazione delle operazioni di creazione e lettura:

### Ereditarietà da CrudRepository

Per attivare le funzionalità viene aggiunta l'annotazione `@Repository`, le interfacce dei repository devono estendere `CrudRepository<T, ID>`, dove `T` è l'entità (es. `AuthorEntity`) e `ID` è il tipo della chiave primaria (es. `Long` o `String`).

```
@Repository
public interface AuthorRepository extends CrudRepository<Author, Long> {
}
```

Non è necessaria alcuna classe di implementazione: Spring genera tutto il codice necessario a runtime.

### Operazione di Creazione: il metodo save()

Nelle implementazioni precedenti con i DAO, avevi un metodo `create()`. Con Spring Data JPA, questo viene sostituito dal metodo `save()`.

- **Funzionamento:** Il metodo `save()` è "intelligente": se l'entità non ha un ID (o l'ID non esiste), esegue un inserimento; se l'ID esiste già, esegue un aggiornamento.
- **Vantaggio del Cascade:** Se hai configurato `CascadeType.ALL` sulla relazione tra Libro e Autore, salvando un oggetto `Book` che contiene un nuovo `Author`, Spring salverà automaticamente entrambi nel database con un'unica operazione.

### Lettura Singola: findById()

L'operazione di recupero di un singolo record (il vecchio `findOne`) viene ora gestita dal metodo `findById(ID id)`.

- Questo metodo restituisce nativamente un `Optional<T>`, integrandosi perfettamente con la gestione moderna dei valori nulli in Java che avevi già impostato nei DAO.

Quindi per il test della creazione di un autore abbiamo dei piccoli cambiamenti al test creato precedentemente:

```
@Test
public void testThatAuthorCanBeCreatedAndRecalled() {
    Author author = TestDataUtil.createTestAuthorA();
    underTest.save(author);
    Optional<Author> result = underTest.findById(author.getId());
    assertThat(result).isPresent();
    assertThat(result.get()).isEqualTo(author);
}
```

Per book invece avremo:

```
@Test
public void testThatBookCanBeCreatedAndRecalled() {
    Author author = TestDataUtil.createTestAuthorA();
    Book book = TestDataUtil.createTestBookA(author);
    underTest.save(book);
    Optional<Book> result = underTest.findById(book.getIsbn());
    assertThat(result).isPresent();
    assertThat(result.get()).isEqualTo(book);
}
```

Inoltre per quanto riguarda la creazione di un libro va fatta una piccola modifica riguardante l'id dell'autore:

```
public static Book createTestBookA(final Author author) {
    return Book.builder()
        .isbn("978-1-2345-6789-0")
        .title("The Shadow in the Attic")
        .author(author)
        .build();
}
```

## Verifica senza SQL

Il video sottolinea che, grazie a questo setup, i test di integrazione passano con successo senza che lo sviluppatore abbia dovuto scrivere una singola query SQL o gestire mappature manuali tramite **RowMapper**. Questa automazione riduce drasticamente il codice "boilerplate".

## Implement Find All

Vediamo adesso come recuperare tutti i record da una tabella utilizzando la potenza di Spring Data JPA, semplificando drasticamente il codice rispetto all'approccio DAO manuale.

### Il Metodo findAll()

Estendendo `CrudRepository`, il tuo repository ottiene automaticamente il metodo `findAll()`. Questo metodo:

- Sostituisce le vecchie query manuali come `SELECT * FROM authors`.
- Restituisce un oggetto di tipo `Iterable<T>` invece di una `List<T>`. Sebbene sia possibile utilizzarlo così com'è, spesso viene convertito in una lista nel Service Layer per una manipolazione più agevole tramite gli Stream di Java.

### Automazione Totale

Il vantaggio principale descritto nel video è che **non è necessario scrivere alcuna riga di SQL** né definire un `RowMapper`. Spring Data JPA analizza l'entità e genera automaticamente la query corretta, mappando ogni colonna del database sui campi dell'oggetto Java in modo trasparente.

### Test di Integrazione

Per verificare il corretto funzionamento, il video mostra come aggiornare i test di integrazione precedentemente scritti per i DAO:

- **Preparazione:** Si salvano più entità nel database utilizzando il metodo `save()` del repository.
- **Esecuzione:** Si richiama `findAll()` e si assegna il risultato a una variabile.
- **Asserzione:** Utilizzando librerie come `AssertJ`, si verifica che la dimensione della collezione sia corretta (es. `hasSize(3)`) e che contenga esattamente gli oggetti salvati.

Quindi per Autori:

```
@Test
public void testThatMultipleAuthorsCanBeCreatedAndRecalled() {
    Author authorA = TestDataUtil.createTestAuthorA();
    underTest.save(authorA);
    Author authorB = TestDataUtil.createTestAuthorB();
    underTest.save(authorB);
    Author authorC = TestDataUtil.createTestAuthorC();
    underTest.save(authorC);

    Iterable<Author> result = underTest.findAll();
    assertThat(result)
        .hasSize(3)
        .containsExactly(authorA, authorB, authorC);
}
```

Mentre per i libri avremo:

```
@Test
public void testThatMultipleBooksCanBeCreatedAndRecalled() {
    Author author = TestDataUtil.createTestAuthorA();

    Book bookA = TestDataUtil.createTestBookA(author);
    underTest.save(bookA);

    Book bookB = TestDataUtil.createTestBookB(author);
    underTest.save(bookB);

    Book bookC = TestDataUtil.createTestBookC(author);
    underTest.save(bookC);

    Iterable<Book> result = underTest.findAll();
    assertThat(result)
        .hasSize(3)
        .containsExactly(bookA, bookB, bookC);
}
```

## Implement Update

A differenza della parte 2 dei database, si mostra come l'aggiornamento dei dati con Spring Data JPA sia estremamente semplificato rispetto al metodo manuale, poiché si basa sulla logica "intelligente" del metodo `save()`.

### Il Metodo "Polimorfico" `save()`

La caratteristica principale di Spring Data JPA è che non esiste un metodo separato per l'aggiornamento: si utilizza lo stesso metodo `save()` impiegato per la creazione. Il framework determina automaticamente l'azione da compiere:

- **Insert:** Se l'entità non ha un ID o l'ID non esiste nel database.
- **Update:** Se l'entità ha un ID già presente, sovrascrivendo i valori della riga corrispondente.

### Workflow di Aggiornamento per l'Autore

Per aggiornare un autore, il processo seguito nei test di integrazione è il seguente:

- **Recupero/Creazione:** Si salva inizialmente un'entità (es. Abigail Rose).
- **Modifica:** Si cambia un attributo dell'oggetto Java, ad esempio impostando il nome su "updated".
- **Salvataggio:** Si richiama `underTest.save(author)`, che esegue l'aggiornamento nel database poiché l'ID è già noto.
- **Verifica:** Si usa `findById()` per recuperare l'autore e si verifica che il nome salvato sia effettivamente quello modificato.

```
@Test
public void testThatAuthorCanBeUpdated() {
    Author authorA = TestDataUtil.createTestAuthorA();
    underTest.save(authorA);

    authorA.setName("UPDATED");
    underTest.save(authorA);

    Optional<Author> result = underTest.findById(authorA.getId());
    assertThat(result).isPresent();
    assertThat(result.get()).isEqualTo(authorA);
}
```

## Workflow per il Libro

Il processo per l'entità **Book** è identico, nonostante utilizzi una stringa (ISBN) come identificatore invece di un numero:

- Si modifica un campo, come il titolo del libro.
- Si richiama il metodo **save()** del repository dei libri.
- Hibernate genera automaticamente la query **UPDATE** corretta per riflettere i cambiamenti.

```
@Test
public void testThatBookCanBeUpdated() {
    Author author = TestDataUtil.createTestAuthorA();

    Book bookA = TestDataUtil.createTestBookA(author);
    underTest.save(bookA);

    bookA.setTitle("UPDATED");
    underTest.save(bookA);

    Optional<Book> result = underTest.findById(bookA.getIsbn());
    assertThat(result).isPresent();
    assertThat(result.get()).isEqualTo(bookA);
}
```

## Vantaggi Architetture

Questo approccio elimina completamente la necessità di scrivere query **UPDATE** manuali e di gestire l'ordine dei parametri SQL, riducendo drasticamente il rischio di errori nel codice.

## Implement Delete

Concludiamo con l'ultimo passaggio per completare le operazioni CRUD di base utilizzando Spring Data JPA.

### Il Metodo deleteById()

L'interfaccia `CrudRepository` fornisce automaticamente il metodo `deleteById(ID id)`. Non è necessario scrivere alcuno script SQL manuale; Spring Data JPA interpreta la chiamata e genera il comando `DELETE` corretto basandosi sulla chiave primaria definita nell'entità (come l'ID per gli autori o l'ISBN per i libri).

### Opzioni di Cancellazione

Il video evidenzia che Spring Data JPA offre diverse flessibilità per la rimozione dei dati:

- **Cancellazione per ID:** Passando direttamente l'identificativo unico del record.
- **Cancellazione per Entità:** È possibile passare l'intero oggetto (es. `delete(authorEntity)`) se è già disponibile nel contesto dell'applicazione.

### Workflow di Test e Verifica

Per garantire che l'operazione funzioni correttamente in un ambiente reale (test di integrazione), la procedura suggerita è la seguente:

- **Salvataggio:** Si crea e si persiste un'entità nel database.
- **Rimozione:** Si esegue il comando di eliminazione tramite il repository.
- **Verifica:** Si tenta di recuperare lo stesso record utilizzando `findById()`. Il test è considerato superato se il risultato restituito è un `Optional.empty`, confermando che il dato non è più presente.

Quindi per l'autore:

```
@Test
public void testThatAuthorCanBeDeleted() {
    Author authorA = TestDataUtil.createTestAuthorA();
    underTest.save(authorA);
    underTest.deleteById(authorA.getId());
    Optional<Author> result = underTest.findById(authorA.getId());
    assertThat(result).isEmpty();
}
```

Mentre per i libri:

```
@Test
public void testThatBookCanBeDeleted() {
    Author author = TestDataUtil.createTestAuthorA();

    Book bookA = TestDataUtil.createTestBookA(author);
    underTest.save(bookA);

    underTest.deleteById(bookA.getIsbn());

    Optional<Book> result = underTest.findById(bookA.getIsbn());
    assertThat(result).isEmpty();
}
```

Semplicità rispetto al DAO

A differenza dell'approccio manuale visto in precedenza, dove occorreva definire la stringa SQL `DELETE FROM table WHERE id = ?` e gestire i parametri tramite `JdbcTemplate`, qui l'operazione viene risolta con una singola riga di codice Java.

Questa sezione chiude le funzionalità CRUD standard generate automaticamente.

## Custom Queries

In questa sezione esploriamo come estendere le funzionalità di Spring Data JPA oltre i metodi CRUD predefiniti, permettendo di creare interrogazioni personalizzate in modo estremamente semplice.

### Query Methods (Derived Queries)

Infatti la funzionalità più sorprendente di Spring Data JPA è la capacità di generare query SQL basandosi esclusivamente sul **nome del metodo** definito nell'interfaccia del Repository.

- **Come funziona:** Il framework analizza il nome del metodo (es. `ageLessThan`) e lo confronta con le variabili d'istanza dell'entità (es. `age`). Se il nome segue le convenzioni di Spring, il framework intuisce la logica e genera automaticamente l'SQL necessario senza che tu debba scrivere nulla.

**Esempio pratico:** Se vuoi trovare tutti gli autori con un'età inferiore a 50 anni, basta definire nel repository:

```
Iterable<AuthorEntity> ageLessThan(int age);
```

- Spring Data JPA capirà autonomamente che deve eseguire una **SELECT** filtrata con una clausola **WHERE age < ?**.

Quindi se volessimo testare ciò avremo:

```
@Test
public void testThatGetAuthorsWithAgeLessThan() {
    Author testAuthorA = TestDataUtil.createTestAuthorA();
    underTest.save(testAuthorA);
    Author testAuthorB = TestDataUtil.createTestAuthorB();
    underTest.save(testAuthorB);
    Author testAuthorC = TestDataUtil.createTestAuthorC();
    underTest.save(testAuthorC);

    Iterable<Author> result = underTest.ageLessThan(50);
    assertThat(result).containsExactly(testAuthorB, testAuthorC);
}
```

Ed all'interno di **AuthorRepository**:

```
@Repository
public interface AuthorRepository extends CrudRepository<Author, Long> {

    Iterable<Author> ageLessThan(int age);

}
```

## HQL (Hibernate Query Language)

Questa sezione approfondisce come gestire query personalizzate che superano le capacità interpretative dei nomi dei metodi di Spring Data JPA.

### Definizione e Differenza con SQL

L'HQL è un linguaggio di interrogazione simile all'SQL, ma con una differenza fondamentale: invece di fare riferimento a tabelle e colonne fisiche del database, l'HQL interroga direttamente le **classi Java (Entity)** e i loro **attributi**. Questo permette di scrivere codice più orientato agli oggetti e meno legato alla struttura specifica del database sottostante.

### Implementazione con @Query

Sebbene i Query Methods siano potenti, hanno dei limiti sintattici. Quando una query è troppo complessa o si desidera utilizzare un nome per il metodo che non segua le convenzioni di Spring, si utilizza l'annotazione **@Query** all'interno dell'interfaccia del Repository.

### Esempio di Sintassi e Parametri

Per una ricerca personalizzata come "autori con età maggiore di", dove il nome del metodo è arbitrario, è necessario definire all'interno di **AuthorRepository**:

```
@Query("SELECT a from Author a where a.age > ?1")
Iterable<Author> findAuthorsWithAgeGreaterThan(int age);
```

- **Mappatura:** **AuthorEntity** si riferisce alla classe Java, non alla tabella SQL.
- **Segnaposti:** Il simbolo **?1** indica che la query deve utilizzare il primo parametro passato al metodo (**int age**) come valore di confronto.

Quindi testando questi parametri:

```
@Test
public void testThatGetAuthorsWithAgeGreaterThan() {
    Author testAuthorA = TestDataUtil.createTestAuthorA();
    underTest.save(testAuthorA);
    Author testAuthorB = TestDataUtil.createTestAuthorB();
    underTest.save(testAuthorB);
    Author testAuthorC = TestDataUtil.createTestAuthorC();
    underTest.save(testAuthorC);

    Iterable<Author> result = underTest.findAuthorsWithAgeGreaterThan(50);
    assertThat(result).containsExactly(testAuthorA);
}
```

## Automazione del Framework

Anche quando si scrive una query in HQL, Spring Data JPA continua a gestire autonomamente la conversione di questo linguaggio in SQL valido per il database in uso, mantenendo tutti i vantaggi dell'automazione vista finora.

Ciò rende il tutto più flessibile dove è possibile passare dai metodi derivati a quelli annotati in base alla complessità della logica richiesta.

# Jackson & JSON

## What is Jackson?

Siamo arrivati ad introdurre la libreria fondamentale che Spring Boot utilizza per gestire il formato JSON, permettendo la comunicazione tra il server Java e il mondo esterno.

### Definizione e Ruolo

**Jackson** è la libreria standard e più popolare per manipolare JSON nel linguaggio Java. Si integra perfettamente con il framework Spring, agendo come un "traduttore" automatico tra gli oggetti Java e le stringhe JSON.

### Concetti Base: Marshalling e Unmarshalling

Il funzionamento di Jackson ruota attorno a due processi bidirezionali:

- **Marshalling (Serializzazione):** Il processo di prendere un oggetto Java (come un'entità **Book**) e trasformarlo in una stringa JSON per inviarlo al client.
- **Unmarshalling (Deserializzazione):** Il processo opposto, in cui Jackson prende una stringa JSON ricevuta in una richiesta (Request Body) e ne inserisce i dati in un oggetto Java utilizzabile dal codice.

### Integrazione in Spring Web

Il video sottolinea che, sebbene sia possibile usare Jackson manualmente, Spring Web lo fa per noi in modo trasparente. Quando un controller restituisce un oggetto, Spring usa Jackson per convertirlo in JSON; quando riceve un JSON, lo converte nell'oggetto richiesto dai parametri del metodo.

Lo strumento principale: `ObjectMapper`

La classe centrale della libreria è l'**`ObjectMapper`**. Per usarla manualmente:

- Si crea un'istanza: `new ObjectMapper()`.
- **`writeValueAsString(obj)`**: Metodo per convertire un oggetto in stringa JSON.
- **`readValue(json, Class)`**: Metodo per leggere un JSON e trasformarlo in un'istanza della classe specificata.

Personalizzazione tramite Annotazioni

Jackson permette di gestire le differenze tra il codice Java e il formato JSON tramite annotazioni sulle classi:

- **`@JsonProperty("nome")`**: Permette di rinominare un campo nel JSON (es. il campo Java `yearPublished` può apparire nel JSON semplicemente come `year`).
- **`@JsonIgnoreProperties(ignoreUnknown = true)`**: Fondamentale per evitare errori se il JSON ricevuto contiene campi extra che non sono presenti nella nostra classe Java.

Questa introduzione a Jackson è il tassello finale prima di costruire l'API REST completa.

## Approfondimento - Testing - Postman e Bruno

Prima di continuare va precisato che da questa sezione in poi verranno effettuati molti test dove è fondamentale utilizzare strumenti software esterni per simulare le richieste dei client (Frontend), testare gli endpoint (**GET**, **POST**, ecc.) e verificare la corretta serializzazione e deserializzazione del JSON effettuata da Jackson.

Ciò perché nello sviluppo di API REST, i controller non restituiscono pagine HTML visualizzabili nel browser.

Nel corso dell'intero video verrà sempre utilizzato Postman, ma più in fondo a questa sottosezione verrà introdotto il suo competitor Bruno, analizzandone i vantaggi in termini di QoL (Quality of Life) e la maggiore leggerezza del software.

### Il flusso di testing con Postman

**Postman** è la piattaforma standard de facto per lo sviluppo e il testing delle API. Nel contesto del codice sopra descritto, Postman agisce come il client e interagisce con Spring Boot attraverso i seguenti passaggi:

#### 1. Test dell'endpoint **GET (/books)**:

- In Postman si imposta il metodo su **GET** e si inserisce l'URL locale (es. **http://localhost:8080/books**).
- Al momento dell'invio, Postman interroga il **BookController**. Jackson effettua il *marshalling* (serializzazione) dell'oggetto **Book** restituito da Spring.
- Postman mostra nel pannello inferiore il payload JSON finale ricevuto e il codice di stato HTTP (**200 OK**).

#### 2. Test dell'endpoint **POST (/books)**:

- In Postman si seleziona il metodo **POST** e lo stesso URL.
- Nella scheda **Body**, si seleziona la voce **raw** e si imposta il formato su **JSON**.
- Si inserisce il testo JSON grezzo
  - i. Es.

```
{
  "isbn": "978-0-13-478627-5",
  "title": "The Enigma of Eternity",
  "author": "Aria Montgomery",
  "yearPublished": "2005"
}
```

- Al clic su **Send**, Postman invia il testo nel corpo della richiesta HTTP. L'annotazione **@RequestBody** intercetta il testo, Jackson esegue l'*unmarshalling* (deserializzazione) nell'oggetto Java **Book**, e il controller stampa i dati nel log di Spring Boot prima di restituire l'oggetto al client.

## Bruno: Un'alternativa moderna e Open Source

Negli ultimi anni, **Bruno** si è affermato come una delle principali alternative a Postman. Sebbene entrambi servano a inviare richieste HTTP e analizzare le risposte JSON di Jackson, Bruno introduce differenze filosofiche e tecniche sostanziali:

- **Architettura dei dati (Git-Friendly):** Postman salva le collezioni di richieste in un formato proprietario o sul proprio cloud, rendendo difficile il tracciamento delle modifiche. Bruno, al contrario, salva ogni singola richiesta come un file di testo puro con estensione `.bru` all'interno di una cartella locale. Questo permette di inserire le richieste di test direttamente nel repository Git del progetto Spring Boot e di condividerle con il team.
- **Privacy e Offline-First:** Postman richiede sempre più spesso la creazione di un account e la sincronizzazione cloud dei dati. Bruno è un client completamente offline ed open-source: non richiede account e garantisce che i dati dei test non lascino mai la macchina locale.
- **Leggerezza:** Bruno si focalizza esclusivamente sul testing delle API senza sovraccaricare l'interfaccia con funzionalità aziendali complesse, risultando significativamente più veloce e leggero nell'utilizzo quotidiano rispetto a Postman.

| Caratteristica       | Postman                                | Bruno                                            |
|----------------------|----------------------------------------|--------------------------------------------------|
| Salvataggio Dati     | Cloud Proprietario / Export JSON       | File locali <code>.bru</code> integrabili in Git |
| Account Obbligatorio | Spesso richiesto per funzioni avanzate | No, 100% Offline e Open Source                   |
| Utilizzo Risorse     | Medio/Alto (Interfaccia complessa)     | Molto Basso (Minimale e mirato)                  |

## Jackson & Spring Web

In questa sezione viene illustrato come **Jackson si integri perfettamente con Spring Web** per gestire la comunicazione dei dati nelle API REST.

### Integrazione Automatica

Jackson agisce come il "motore" invisibile dietro Spring Web. Quando crei un progetto con la dipendenza `spring-boot-starter-web`, Spring Boot configura automaticamente Jackson per gestire la traduzione tra oggetti Java e formato JSON.

Nel nostro setup, il controller sarà:

```
@RestController
@Log
public class BookController {
    @GetMapping(path = "/books")
    public Book retrieveBook() {
        return Book.builder()
            .isbn("978-0-13-478627-5")
            .title("The Enigma of Eternity")
            .author("Aria Montgomery")
            .yearPublished("2005")
            .build();
    }

    @PostMapping(path = "/books")
    public Book createBook(@RequestBody final Book book) {
        Log.info("Got book: " + book.toString());
        return book;
    }
}
```

Mentre la classe book sarà un semplicissimo oggetto:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class Book {

    private String isbn;

    private String title;

    private String author;

    private String yearPublished;
}
```

## Funzionamento nelle Richieste GET (Marshalling)

Quando un client richiede dati (ad esempio tramite un endpoint `GET /books`), il processo è il seguente:

- **Restituzione Oggetto:** Il metodo del controller restituisce un oggetto Java puro (un POJO come `Book`).
- **Conversione:** Spring rileva che il client si aspetta una risposta JSON e utilizza Jackson per serializzare l'oggetto Java in una stringa JSON.
- **Risultato:** Il client riceve il JSON nel corpo della risposta senza che tu debba scrivere codice di conversione manuale.

## Funzionamento nelle Richieste POST (Unmarshalling)

Per inviare dati al server (ad esempio per creare un nuovo libro tramite `POST /books`), si utilizza l'annotazione `@RequestBody`:

- **Ricezione JSON:** Il client invia una stringa JSON nel corpo della richiesta.
- **Conversione:** Grazie a `@RequestBody`, Spring passa quel JSON a Jackson, che lo deserializza istantaneamente in un oggetto Java pronto per essere usato dal tuo codice.
- **Trasparenza:** Questo processo avviene automaticamente basandosi sui nomi dei campi; se il JSON ha un campo `title`, Jackson cercherà un attributo `title` nella classe Java corrispondente.

## Semplicità del Codice

Il video sottolinea che non è necessaria alcuna configurazione speciale sulle classi di dominio (Entity o DTO) per attivare questa conversione. Finché la classe Java segue le convenzioni (come avere i getter/setter o usare Lombok), Jackson sarà in grado di mappare i dati correttamente.

## Java Objects to JSON

Qui parleremo dell'utilizzo pratico della classe **ObjectMapper** di Jackson per convertire manualmente gli oggetti Java in stringhe JSON.

### Istanza di ObjectMapper

Per iniziare la conversione, è necessario creare un'istanza della classe principale di Jackson, l'**ObjectMapper**. In un contesto di test o di utilità manuale, si può istanziare semplicemente con `new ObjectMapper()`.

### Il Metodo writeValueAsString

Il metodo chiave per trasformare un oggetto Java in JSON è **writeValueAsString(Object value)**.

- **Input:** Accetta qualsiasi "Plain Old Java Object" (POJO), come la classe **Book** usata nell'esempio.
- **Output:** Restituisce una stringa che rappresenta l'oggetto in formato JSON.

```
public class JacksonTests {  
  
    @Test  
    public void testThatObjectMapperCanCreateJsonFromJavaObject() throws  
        JsonProcessingException {  
        ObjectMapper objectMapper = new ObjectMapper();  
        Book book =  
            Book.builder()  
                .isbn("978-0-13-478627-5")  
                .title("The Enigma of Eternity")  
                .author("Aria Montgomery")  
                .yearPublished("2005")  
                .build();  
  
        String result = objectMapper.writeValueAsString(book);  
        assertThat(result)  
            .isEqualTo(  
                "{ \"isbn\": \"978-0-13-478627-5\", \"title\": \"The Enigma of  
Eternity\", \"author\": \"Aria  
                + \" Montgomery\", \"year\": \"2005\" }\" );  
    }  
}
```

## Gestione delle Eccezioni

Un aspetto critico sottolineato nel video è che il metodo `writeValueAsString` può sollevare una **`JsonProcessingException`**. Questa eccezione deve essere obbligatoriamente gestita tramite un blocco `try-catch` o dichiarata nella firma del metodo (clausola `throws`), altrimenti il codice non compilerà.

## Mappatura dei Campi

Per impostazione predefinita, Jackson crea le chiavi del JSON basandosi esattamente sui nomi delle variabili d'istanza della classe Java. Ad esempio, se la classe ha un campo `title`, nel JSON apparirà `"title": "valore"`.

## Verifica nei Test

Il video mostra come verificare questa conversione tramite un'asserzione (es. con AssertJ): si confronta la stringa generata dall'`ObjectMapper` con una stringa JSON di riferimento per assicurarsi che tutti i campi siano stati serializzati correttamente.

## JSON to Java Objects

Questa sezione approfondisce il processo inverso della serializzazione, noto come **deserializzazione** o **unmarshalling**, ovvero la conversione di una stringa JSON in un oggetto Java.

### Il Metodo `readValue()`

Per trasformare il JSON in un oggetto, la classe `ObjectMapper` mette a disposizione il metodo `readValue()`. La sintassi richiede due parametri:

- La stringa JSON da convertire.
- Il riferimento alla classe di destinazione (es. `Book.class`).

### Requisiti della Classe Java

Affinché Jackson possa ricostruire l'oggetto, la classe Java deve possedere un **costruttore senza argomenti** (no-args constructor). Jackson lo utilizza per istanziare l'oggetto prima di popolarne i campi tramite i metodi setter.

```
@Test
public void testThatObjectMapperCanCreateJavaObjectFromJsonObject()
    throws JsonProcessingException {
    Book book =
        Book.builder()
            .isbn("978-0-13-478627-5")
            .title("The Enigma of Eternity")
            .author("Aria Montgomery")
            .yearPublished("2005")
            .build();
    String json =
        "{ \"isbn\": \"978-0-13-478627-5\", \"title\": \"The Enigma of\"
        + \" Eternity\", \"author\": \"Aria
        Montgomery\", \"yearPublished\": \"2005\" }";

    final ObjectMapper objectMapper = new ObjectMapper();

    Book result = objectMapper.readValue(json, Book.class);
    assertThat(result).isEqualTo(book);
}
```

## Logica di Mappatura e Convenzioni

- **Corrispondenza Nomi:** Jackson tenta di far corrispondere automaticamente le chiavi del JSON con i nomi delle variabili d'istanza della classe Java (es. la chiave "title" viene mappata sul campo `title`).
- **Annotazione `@JsonProperty`:** Se i nomi non corrispondono, puoi usare questa annotazione sul campo Java per istruire Jackson su quale chiave cercare nel JSON.

## Gestione degli Errori e Campi Sconosciuti

- **Eccezioni:** Il processo può generare una `JsonProcessingException`, che deve essere gestita obbligatoriamente nel codice.
- **Campi Extra:** Se il JSON contiene chiavi che non esistono nella classe Java, Jackson solleverà un'eccezione `UnrecognizedPropertyException`.
- **Soluzione:** Per rendere l'applicazione più flessibile e ignorare i campi non necessari, si applica l'annotazione `@JsonIgnoreProperties(ignoreUnknown = true)` a livello di classe.

## Renaming JSON Properties

Vediamo invece come gestire i casi in cui i nomi dei campi nel tuo codice Java non corrispondono ai nomi desiderati nel formato JSON, utilizzando la personalizzazione offerta dalla libreria Jackson.

### La convenzione di default

Normalmente, Jackson mappa le chiavi JSON basandosi esattamente sui nomi delle variabili d'istanza della classe Java. Ad esempio, un campo chiamato `yearPublished` in Java apparirebbe come `"yearPublished"` nel file JSON di default.

### L'annotazione `@JsonProperty`

Per cambiare questo comportamento e rinominare una proprietà, si utilizza l'annotazione `@JsonProperty` direttamente sopra il campo nella classe Java.

- **Sintassi:** Si passa come argomento la stringa con il nome desiderato per il JSON, ad esempio `@JsonProperty("year")`.

**Esempio del video:** Se si desidera che il campo Java `yearPublished` venga visualizzato nel JSON semplicemente come `"year"`, si annota il campo all'interno di `Book` così:

```
@JsonProperty("year")
private String yearPublished;
```

### Effetto Bidirezionale

L'uso di questa annotazione ha effetto su entrambi i processi gestiti da Jackson:

- **Serializzazione:** Quando trasformi l'oggetto Java in JSON, Jackson userà il nome specificato nell'annotazione.
- **Deserializzazione:** Quando ricevi un JSON, Jackson cercherà la chiave specificata nell'annotazione per popolare il campo Java corrispondente.
- Quindi avremo una stringa per identificare il JSON come:

```
{"isbn\":\"978-0-13-478627-5\",\"title\":\"The Enigma of Eternity\",\"author\":\"Aria Montgomery\",\"year\":\"2005\"}"
```

### Integrazione con Spring Web

Tutte le personalizzazioni fatte tramite `@JsonProperty` vengono automaticamente rispettate dall'interfaccia di Spring Web. Questo significa che i tuoi endpoint REST utilizzeranno i nomi rinominati senza bisogno di ulteriori configurazioni manuali nei Controller.

Questa funzionalità è fondamentale per mantenere il codice Java pulito e conforme alle proprie convenzioni di naming, pur rispettando i requisiti di formato di un'API esterna.

## Ignore Properties

Affrontiamo adesso un problema comune nelle API REST: come comportarsi quando si riceve un JSON che contiene dati "extra" o non previsti dalla classe Java corrispondente.

Il problema: `UnrecognizedPropertyException`

Per impostazione predefinita, Jackson è molto rigoroso durante la fase di deserializzazione. Se riceve un JSON contenente un campo che non è stato definito come variabile d'istanza nella classe Java (ad esempio un campo di test come `"foo": "bar"`), Jackson interrompe il processo lanciando un'eccezione di tipo **`UnrecognizedPropertyException`**, facendo fallire la richiesta.

```
@Test
public void testThatObjectMapperCanCreateJavaObjectFromJsonObject()
    throws JsonProcessingException {
    Book book =
        Book.builder()
            .isbn("978-0-13-478627-5")
            .title("The Enigma of Eternity")
            .author("Aria Montgomery")
            .yearPublished("2005")
            .build();
    String json =
        "{\"foo\":\"bar\", \"isbn\":\"978-0-13-478627-5\", \"title\":\"The Enigma of"
        + " Eternity\", \"author\":\"Aria Montgomery\", \"year\":\"2005\"}";

    final ObjectMapper objectMapper = new ObjectMapper();

    Book result = objectMapper.readValue(json, Book.class);
    assertThat(result).isEqualTo(book);
}
```

La soluzione: `@JsonIgnoreProperties`

Per rendere l'applicazione più robusta e flessibile rispetto a cambiamenti improvvisi nei dati in ingresso, si utilizza l'annotazione **`@JsonIgnoreProperties`** posta direttamente sopra la definizione della classe.

Configurazione `ignoreUnknown = true`

L'attributo fondamentale da impostare all'interno dell'annotazione è **`ignoreUnknown = true`** (il cui valore predefinito è invece `false`).

- **Effetto:** Questa istruzione dice a Jackson di ignorare silenziosamente qualsiasi proprietà presente nel JSON che non trovi una corrispondenza esatta nei campi della classe Java.

- **Utilità:** È una configurazione essenziale quando si lavora con API di terze parti o quando si vuole garantire che l'applicazione non si rompa se il client invia accidentalmente dati aggiuntivi non necessari.
- Quindi otterremo l'oggetto book:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
@JsonIgnoreProperties(ignoreUnknown = true)
public class Book {

    private String isbn;

    private String title;

    private String author;

    @JsonProperty("year")
    private String yearPublished;
}
```

Questa impostazione conclude la panoramica sulle personalizzazioni fondamentali di Jackson per la gestione del formato JSON.